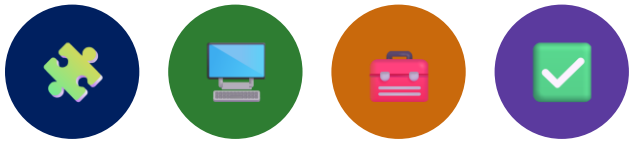
"Good automation is reliable, maintainable, and provides confidence in every release."

MODULE 19

Integration Testing and UI Test Automation

Validate the application as a whole and automate real user interactions against the Angular UI.



TEKsystems Global Services

WORKFORCE DEVELOPMENT

MODULE 19 OVERVIEW

Validate the application as a whole, then automate real user interactions with Selenium against the Angular UI.

Unit tests verify pieces in isolation, but real confidence comes from testing components working together and validating the UI a user actually sees -- this module covers integration testing, Spring Test, and Selenium against the Northstar CRM Angular front end.

DURATION & LAB

Theory

Integration testing, the Spring Test framework, and Selenium WebDriver fundamentals.

Hands-On Practice

Spring Boot integration tests plus Selenium automation against a real running Angular app.

Scenario

Regression-test the Northstar CRM app end-to-end, from the Spring Boot API through the Angular UI.

MODULE ARC

Integration Testing

Validate components working together -- controllers, services, repositories, and a real database.

UI Test Automation

Automate real user workflows against the Angular UI with Selenium WebDriver.

The Payoff

Quality, reliability, and faster delivery -- confidence that the whole system, not just its pieces, works.

KEY TAKEAWAY Good automation is reliable, maintainable, and provides confidence in every release -- integration tests prove the layers fit together, and Selenium proves the Angular UI a user actually sees works too.

WHY INTEGRATION TESTING MATTERS

Unit tests pass, but the application can still fail when components are wired together.

Catch Integration Bugs

Find issues that only appear when components interact -- a mocked repository can hide a failure a real one would catch.

Validate Real Wiring

Verify controllers, services, and repositories genuinely work together, not just that each compiles and passes alone.

Test Real Protocols

Exercise HTTP, database, and messaging boundaries -- the places where unit-test mocks quietly hide real behavior.

KEY TAKEAWAY Integration tests catch the bugs that unit tests can't -- the ones that only appear when components work together.

Why Integration Testing Matters

Ensure modules work together as expected

Validates Component Interactions

Ensures that integrated modules, services, and APIs communicate and behave correctly together.

Finds Issues Earlier

Detects defects in interfaces, data flow, configurations, and contracts before they reach production.

Prevents Cascading Failures

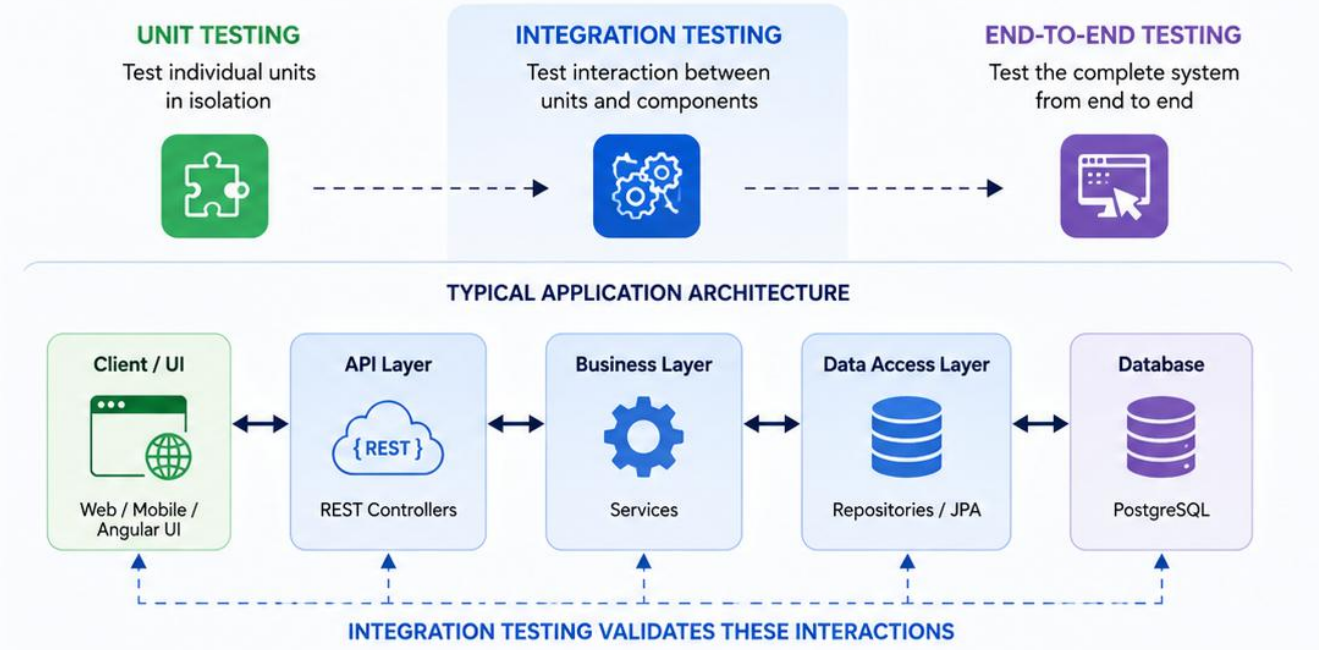
Catches problems that can cause multiple components to fail in production.

Improves System Reliability

Builds confidence that the system works as a whole, not just in isolated parts.

Saves Time and Cost

Fixing issues earlier is faster and cheaper than debugging in later environments.



BUSINESS IMPACT

Higher Quality
Fewer defects in production

Faster Releases
Confident deployments with less rework

Better User Experience
Smooth interactions across the application

Reduced Risk
Protects business from system failures

Lower Cost
Detect issues early, save time and money

MODULE LEARNING OBJECTIVES

By the end of this module, you will be able to:

1. Distinguish unit, integration, and end-to-end testing by scope, speed, and purpose, and place each on the test pyramid.
2. Write Spring Boot integration tests using @SpringBootTest, test slices, and the right web-environment option for each boundary.
3. Test controllers with MockMvc, repositories against a real PostgreSQL instance, and services with a partially real context.
4. Choose a deliberate test database strategy and manage test data and isolation so tests are repeatable.
5. Automate real user journeys against the Angular UI using Selenium WebDriver and stable, resilient element selectors.
6. Write explicit waits that account for Angular's asynchronous rendering, instead of brittle fixed sleeps.
7. Run a headless Selenium suite inside a GitHub Actions workflow, capturing screenshots and reports on failure.
8. Recognize common causes of flaky UI tests and apply best practices that keep an automation suite reliable over time.

KEY TAKEAWAY These eight objectives take you from testing Spring Boot components in isolation to proving the whole Northstar CRM system -- API and Angular UI together -- works end to end, automatically, in CI.

Learning Objectives

By the end of this module, you will be able to:

01 **Understand Integration Testing**
Explain the purpose and benefits of integration testing in enterprise applications.

02 **Apply Spring Boot Integration Testing**
Use @SpringBootTest and test slices to test different layers of a Spring Boot application.

03 **Manage Test Data and Databases**
Configure and manage PostgreSQL test databases, ensure test isolation, and load test data effectively.

04 **Automate UI Tests with Selenium**
Use Selenium WebDriver to automate Angular application UI tests reliably.

05 **Test Angular UIs Effectively**
Identify stable selectors, handle Angular rendering, and test user interactions and navigation flows.

06 **Improve Test Reliability**
Use explicit waits, handle loading states, and avoid flaky tests using best practices.

07 **Integrate Tests in CI/CD Pipelines**
Execute automated tests in GitHub Actions and publish reports and artifacts.

08 **Follow Enterprise Testing Best Practices**
Apply a robust testing strategy to deliver high-quality, reliable, and maintainable applications.

Outcome

You will be able to design, implement, and automate integration and UI tests to ensure application quality across the full stack.

Reliable Applications | Faster Releases | Higher Quality | Stronger Confidence

TESTING STRATEGY OVERVIEW

A deliberate mix of test levels, not just 'more tests,' is what actually builds shippable confidence.

Fast Feedback First

Many fast unit tests catch logic errors in seconds -- run them on every save, not just before a commit.

Fewer, Slower Integration Checks

Integration tests confirm the wiring at real boundaries -- run them before every merge, not on every keystroke.

Fewest, Slowest End-to-End Runs

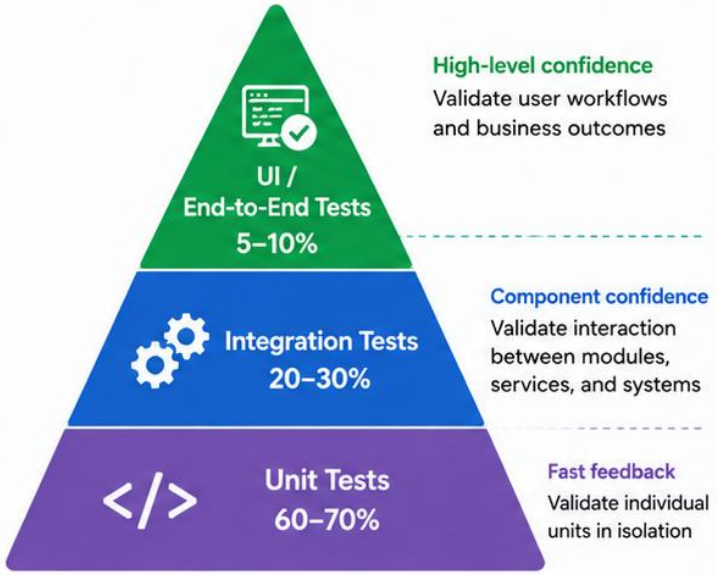
Selenium-driven E2E tests prove critical user journeys work through the deployed stack -- reserve them for what truly matters.

KEY TAKEAWAY A strategy is a deliberate shape, not an accident -- lots of fast tests at the bottom, fewer slower ones as you go up, each level catching what the level below it structurally cannot.

Testing Strategy Overview

A layered and balanced approach to deliver high-quality, reliable software

TEST AUTOMATION PYRAMID



High-level confidence
Validate user workflows and business outcomes

Component confidence
Validate interaction between modules, services, and systems

Fast feedback
Validate individual units in isolation

OUR TESTING STRATEGY

Quality First
Build quality in early and test continuously.

Layered Testing
Combine unit, integration, and UI tests for comprehensive coverage.

Automation First
Automate as much as possible to increase speed and reliability.

Risk-Based Approach
Focus testing effort on critical business flows and high-risk areas.

Continuous Feedback
Provide fast feedback to developers and stakeholders.

TEST TYPES AND SCOPE

Test Type	Purpose	Scope / Example
Unit Testing	Validate individual functions / classes in isolation	• Service methods • Utility functions • Validators
Integration Testing	Validate interactions between components and external systems	• REST APIs • Database • Messaging (Kafka)
UI / End-to-End Testing	Validate complete user journeys in the application	• Login flow • CRUD workflows • Navigation
Non-Functional Testing	Validate quality attributes of the application	• Performance • Security • Reliability

TESTING BEST PRACTICES

- Automate repetitive and high-value tests
- Use stable test data and maintain test independence
- Fail fast and provide clear, actionable feedback
- Monitor test results and continuously improve
- Collaborate across development, QA, and operations

UNIT VS INTEGRATION VS END-TO-END TESTING

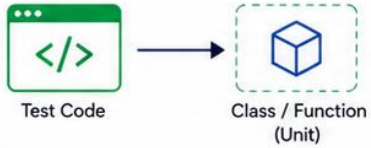
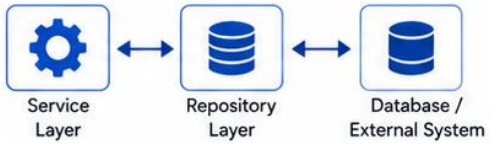
Three levels, three different things being verified -- all essential, none a substitute for the others.

ASPECT	UNIT	INTEGRATION	END-TO-END
Scope	A single class or method	Multiple components together	The full running application
Dependencies	Mocked/stubbed	Real at the boundary; unrelated externals replaced	Real, through external interfaces
Speed	Very fast	Slower than unit tests	Slowest of the three
Purpose	Verify business logic correctness	Verify components integrate correctly	Verify a real user journey works
Typical Tools	JUnit 5, Mockito	Spring Test, @SpringBootTest, Testcontainers	Selenium WebDriver

KEY TAKEAWAY Unit tests build confidence in the code you write; integration tests build confidence in the system you deliver; end-to-end tests build confidence in the journey a user actually takes.

Unit vs Integration vs End-to-End Testing

Different scopes. Different goals. Complementary for high-quality software.

	UNIT TESTING	INTEGRATION TESTING	END-TO-END TESTING
	Test individual units in isolation	Test interactions between components	Test the complete system from end to end
			
FOCUS	Validate individual functions / methods / classes	Validate interactions between modules, services, and external systems	Validate complete user workflows and business outcomes
SCOPE	Smallest scope No dependencies (mocks / stubs used)	Medium scope Real dependencies (DB, APIs, Messaging, etc.)	Largest scope Entire application stack
EXAMPLE	Test a service method, utility function, or business rule	Test service + repository with a real database, or API + external service	User logs in, creates an employee, and receives confirmation
TOOLS	JUnit, Mockito	Spring Boot Test, Testcontainers, RestAssured	Selenium, Cypress, Playwright, Postman
SPEED	Very Fast	Moderate	Slow
COST	Low	Moderate	High
FEEDBACK	Immediate feedback to developers	Fast feedback on integration issues	Feedback on production-like behavior

KEY TAKEAWAY

Use all three types together. Unit tests ensure correctness, integration tests ensure components work together, and end-to-end tests ensure the system delivers value to the user.

Better Quality. Higher Confidence.

ENTERPRISE TEST PYRAMID

Many fast unit tests at the base, fewer integration tests in the middle, fewest and slowest E2E tests at the top.

Wide Base: Unit Tests

The majority of the suite -- fast, cheap, and run constantly, catching logic errors the instant they're introduced.

Middle: Integration Tests

Fewer in number, confirming real components actually work together at real boundaries -- database, HTTP, messaging.

Narrow Top: E2E Tests

Fewest and slowest -- Selenium-driven journeys reserved for the Northstar CRM's most critical user flows only.

KEY TAKEAWAY An inverted pyramid -- mostly slow E2E tests and few unit tests -- is a well-known anti-pattern: it's expensive to run, slow to give feedback, and brittle to maintain.

TRY IT YOURSELF — EXERCISE 1: PLACE TESTS ON THE PYRAMID

Reinforces: the unit-versus-integration-versus-end-to-end distinction and the enterprise test pyramid you just covered.

STEPS (notes/lab19-test-level-map.md)

Step	What To Do
1 — Three Levels	Give one CRM example at each level: unit, integration, end-to-end.
2 — Cost and Speed	For each level, note roughly how fast it runs and what it can prove.
3 — Shape	Say why there are many unit tests and few end-to-end ones.
4 — Scope	Plan only — no test code written in this pre-lab.

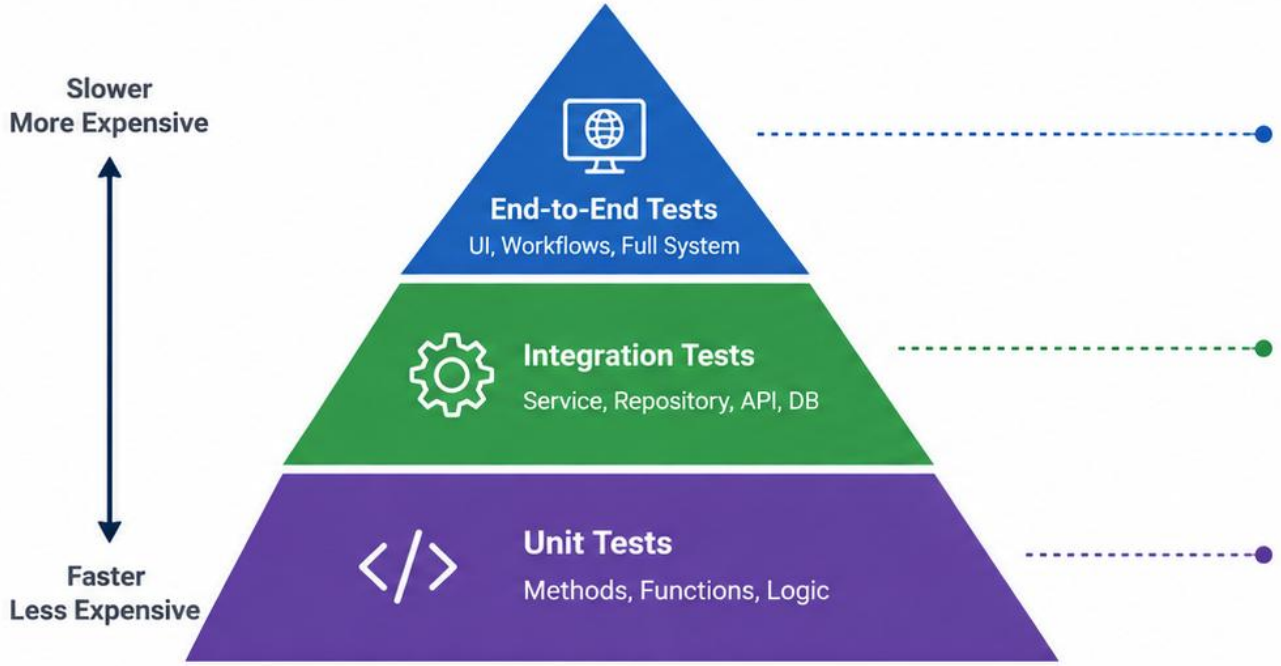
Pyramid for the CRM

```
unit        CustomerService status rule        milliseconds
integration GET /api/customers/CUS-1001 + DB   seconds
e2e         Selenium: log in, find Amina, edit  tens of seconds
```

TRY IT NOW — Complete Exercise 1 before continuing: exercises/exercise-01-test-level-map.md (workspace: examples/module-19-exercises).

Enterprise Test Pyramid

The Enterprise Test Pyramid is a strategic testing model that balances different types of tests to achieve high confidence with optimal cost and speed.



Few in Number

Validate complete user journeys and critical business workflows.

Examples: Selenium UI tests, API workflows

Moderate in Number

Verify interactions between components and external dependencies.

Examples: Spring Boot integration tests, repository tests, API tests

Many in Number

Validate individual units of code in isolation for correctness and reliability.

Examples: JUNit tests, component tests

Goal:
Maximize test coverage and confidence while minimizing time and cost.

- **Higher Quality** — Catch defects early
- **Faster Feedback** — Shorter development cycle
- **Lower Cost** — Fix defects when cheaper
- **Greater Confidence** — Deliver reliable software

Balance is key: focus on unit tests, strengthen integration tests, and cover critical end-to-end scenarios.

SPRING BOOT INTEGRATION TESTING

Different test boundaries use different runtime models, from a web-layer slice to a full HTTP round trip.

1 Web Slice (MockMvc)

Test -> MockMvc -> Controller, with no running server -- fast, focused checks on request mapping and validation.

2 Full HTTP Integration

TestRestTemplate -> Running Spring Boot Server -> Service -> Repository -> Test DB, exactly like a real client.

3 Testcontainers-Backed

@Testcontainers boots a real PostgreSQL instance so repository tests run against production-like SQL.

KEY TAKEAWAY Integration tests trade some speed for confidence that real components genuinely work together.

SPRING BOOT INTEGRATION TESTING

Integration testing validates how multiple components work together in a Spring Boot application, including layers, configurations, and external dependencies.

KEY FEATURES

Full Application Context
Loads the full Spring Boot context to test components together.

Realistic Integration
Verifies interactions between layers (Controller, Service, Repository, DB, etc.).

Configurable Test Scope
Use test slices for focused testing or load full context as needed.

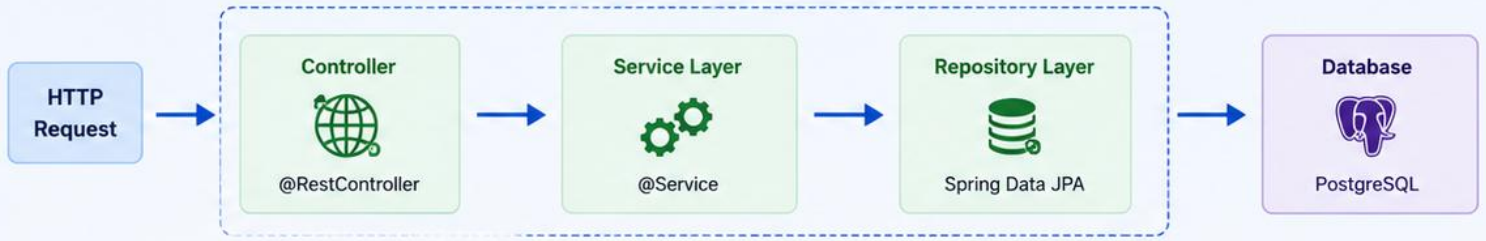
Reliable & Reproducible
Uses test-specific configuration and data for consistent results.

Supports External Dependencies
Test with embedded DB, Testcontainers, or mocked services.

WHEN TO USE

- Validate interactions across layers
- Verify configurations & auto-configurations
- Test with real database or Testcontainers
- Ensure component wiring is correct
- Validate transactions and data integrity

INTEGRATION TESTING SCOPE



TEST CONTEXT COMPONENTS

EXAMPLE: @SpringBootTest

```
@SpringBootTest
@AutoConfigureMockMvc
class EmployeeControllerIntegrationTest {
   @Autowired
   private MockMvc mockMvc;
   @Test
   void should_return_employee_list() throws Exception {
      mockMvc.performm(get("/api/employees"))
         .andExpect(status().isOk())
         .andExpect(jsonPath("$.data").isArray());
   }
}
```

COMMON TEST ANNOTATIONS

@SpringBootTest	Loads full application context.
@WebMvcTest	Slice test for web layer (controllers, filters, etc.).
@DataJpaTest	Slice test for JPA repositories.
@TestConfiguration	Provide custom beans for tests.

BEST PRACTICES

- Use Test-Specific Data Sources
- Isolate Data Between Tests
- Use @DirtiesContext When Needed
- Keep Tests Fast and Focused
- Balance Coverage and Performance

@SPRINGBOOTTEST

Four web-environment options decide whether the test starts a real server at all.

WEB ENVIRONMENT	BEHAVIOR
MOCK (default)	A mock servlet environment -- no real HTTP port is opened.
RANDOM_PORT	Starts a real embedded server on a random free port -- used for TestRestTemplate-based HTTP tests.
DEFINED_PORT	Starts a real server on the port configured in application properties.
NONE	Loads the ApplicationContext with no web environment at all.
Typical use	RANDOM_PORT + TestRestTemplate is the pattern this course's CustomerApiIT-style tests use.

KEY TAKEAWAY @SpringBootTest with RANDOM_PORT turns an integration test into a real HTTP call against a genuinely running server -- exactly like curl or a browser would make.

@SpringBootTest

@SpringBootTest loads the entire Spring Boot application context for integration testing. It is the starting point for end-to-end testing of your application layers together.

Key Purpose

- Loads the full Spring Application Context
- Starts an embedded web server (by default)
- Auto-configures all beans, components, and settings
- Ideal for end-to-end integration tests
- Supports real HTTP calls, databases, messaging, etc.

Java Test

```
@SpringBootTest
@AutoConfigureMockMvc
class EmployeeIntegrationTest {

   @Autowired
   private MockMvc mockMVc;

   @Test
   void shouldReturnAllEmployees() throws Exception {
      mockMvc.perform(get("/api/employees"))
             .andExpect(status().isOk())
             .andExpect(jsonPath("$.length()").isNumber());
   }
}
```

Starts Embedded Server | Real Database (Test Profile) | All Beans & Auto-Configuration | End-to-End Integration Tests | Works with MockMvc

Best Practices

- Use @SpringBootTest for tests that span multiple layers (web, service, repository).
- Prefer test slices (@WebMvcTest, @DataJpaTest) for focused, faster tests.
- Use @SpringBootTest only when the full context is required.

Tip

Combine @SpringBootTest with @AutoConfigureMockMvc for powerful controller integration testing without starting a real server.

TEST SLICES

A slice test loads only the part of the Spring context relevant to one layer -- faster than a full context.

ANNOTATION	PURPOSE	TYPICAL USE
@WebMvcTest	Spring MVC test slice: request mapping, serialization, validation, web-layer behavior.	Testing controllers in isolation
@DataJpaTest	JPA/repository layer; may substitute an embedded DB unless configured otherwise.	Testing repository queries
@AutoConfigureMockMvc	Auto-configures MockMvc for web-layer tests.	REST endpoints, no running server
@MockBean	Replaces a context collaborator with a test double.	Excluding one collaborator's real behavior
@ActiveProfiles / @TestPropertySource	Selects test config / overrides individual properties.	Pointing config at a test database

KEY TAKEAWAY Choose the narrowest Spring Test annotation that gives you real confidence -- full-context tests are powerful but slower than a well-chosen slice.

TEST SLICES

Test slices load only the specific part of the application context needed for a test.
They provide faster tests with minimal dependencies and focused validation.

WHY USE TEST SLICES?

Faster Execution
Loads only the required components.

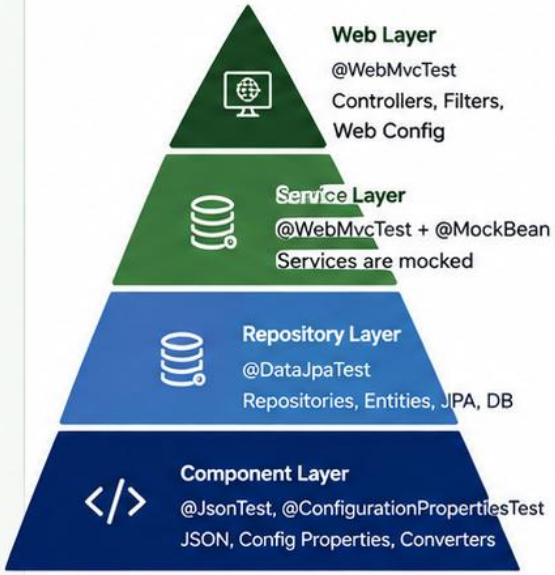
Focused Testing
Tests a specific layer or concern in isolation.

Minimal Dependencies
Auto-configures only what the slice needs.

Less Resource Usage
No need to start the entire application context.

Better Test Maintainability
Clear intent and smaller test scope.

TEST SLICE PYRAMID



POPULAR TEST SLICES

Annotation	Focus Area	Loaded Components	Use When
@WebMvcTest	Web Layer	Controllers, MVC config, filters, Jackson, validation	Testing controllers, request mapping, validation, etc.
@DataJpaTest	JPA Layer	Repositories, Entities, JPA config, Test Entity Manager, DataSource	Testing repositories, queries, custom methods
@WebFluxTest	Reactive Web Layer	WebFlux controllers, handlers, filters, config	Testing reactive controllers
@JsonTest	JSON Layer	Jackson ObjectMapper, JSON serializers/ deserializers	Testing JSON serialization and deserialization
@ConfigurationPropertiesTest	Config Properties	Configuration properties classes, validation	Testing @Configuration Properties classes

BEST PRACTICES

- Use the smallest slice that validates your requirement.
- Mock external dependencies with @MockBean.
- Use @DataJpaTest with an in-memory database for speed.
- Keep tests isolated and independent.
- Avoid loading the full context unless necessary.
- Combine slices thoughtfully when needed.

EXAMPLE: @WebMvcTest

```
@WebMvcTest(EmployeeController.class)
class EmployeeControllerTest {
    @Autwwired
    private MockMvc mockMvc;

    @MockBean
    private EmployeeService employeeService;
}
```

- Loads only web components.
- Services are mocked.
- Fast and focused controller tests.
- Perfect for request mapping, validation, and error handling.

TAKEAWAY: Test slices help you write fast, reliable, and maintainable tests by loading only what you need.

spring boot

CONTROLLER TESTING

@WebMvcTest loads only the web layer -- MockMvc drives requests without a running server.

Web Layer Only

@WebMvcTest(CustomerController.class) loads just the controller under test, not the full application context.

@MockBean the Service

The real CustomerService is replaced with a mock, so only the controller's own logic and mapping are exercised.

One Fluent Assertion

mockMvc.perform(...) chains status, header, and JSON body checks in a single fluent call.

KEY TAKEAWAY MockMvc gives fast, focused controller tests without paying the cost of a full running server.

CONTROLLER TESTING

Controller testing validates the web layer of a Spring Boot application in isolation to ensure request mapping, input handling, validation, and responses work as expected.

WHAT IS CONTROLLER TESTING?

Tests the REST controllers without starting the full application context. Dependencies such as services are mocked.

Focus on Web Layer
Validate endpoints, request mapping, headers, parameters, and responses.

Fast and Lightweight
Loads only the web layer using MockMvc.

Reliable and Isolated
Services are mocked to isolate the controller behavior.

HOW IT WORKS

TYPICAL TEST FLOW

COMMON TEST SCENARIOS

- Validate HTTP status codes
- Validate JSON response body
- Validate path variables
- Validate query parameters
- Validate request body (JSON)
- Validate validation errors
- Validate response headers

EXAMPLE TEST WITH MOCKMVC

```
@WebMvcTest(EmployeeController.class)
class EmployeeControllerTest {
   @Autowired
   private MockMvc mockMvc;

   @MockBean
   private EmployeeService employeeService;

   @Test
   void should_return_employee_by_id() throws Exception {
      Employee emp = new Employee(1L, "John Doe", "john@example.com");
      when(employeeService.getById(1L)).thenReturn(emp);

      mockMvc.perform(get("/api/employees/1"))
           .andExpect(status().isOk())
           .andExpect(jsonPath("$.id").value(1))
           .andExpect(jsonPath("$.name").value("John Doe"));
}
```

ANNOTATIONS USED

@WebMvcTest	Loads only web layer components.
@MockBean	Creates a mock for service/repository beans.
MockMvc	Performs HTTP requests and assertions.
@Autowired	Injects MockMvc for testing.
@Test	Marks the test method.

KEY MOCKMVC ASSERTIONS

`status().isOk()`	Asserts HTTP 200 OK
`status().isBadRequest()`	Asserts HTTP 400 Bad Request
`status().isNotFound()`	Asserts HTTP 404 Not Found
`jsonPath("$.field").value(x)`	Asserts JSON field value
`content().contentType(...)`	Asserts response content type
`header().string("X-Total", "10")`	Asserts response headers

BEST PRACTICE: Test one scenario per test method, keep tests focused and independent, and mock only what is necessary.

spring boot

REPOSITORY INTEGRATION TESTING

@Testcontainers starts a real database in Docker so repository tests run against production-like SQL.

Real PostgreSQL, Not H2

A static PostgreSQLContainer boots a genuine Postgres instance for the test run, in Docker.

@DynamicPropertySource

Wires the container's live JDBC URL and credentials into the Spring context before it starts.

Catches Real SQL Issues

Database-specific constraints, functions, and query behavior that an in-memory database would miss or mishandle.

KEY TAKEAWAY Testcontainers trades a little speed for confidence that queries hit a real database engine, not an approximation of one.

REPOSITORY INTEGRATION TESTING

Repository integration testing verifies that Spring Data JPA repositories interact correctly with the database, execute queries as expected, and return the right results.

WHY REPOSITORY INTEGRATION TESTING?

Validate Data Access Logic
Ensure queries, relationships, and entity mappings work with a real database.

Detect Query Issues Early
Catch JPQL, native query, and mapping issues before runtime.

Use Real Database Behavior
Test against PostgreSQL to verify constraints, indexes, and transactions.

Reliable and Repeatable
Controlled test data and rollbacks ensure consistent test outcomes.

TEST ARCHITECTURE

WHAT @DATAJPATEST PROVIDES

- Configures JPA components
- In-memory or Testcontainers DB
- Scans @Entity classes
- Configures repositories
- Transactional tests (rollback)

WHAT WE TEST

- CRUD operations
- Derived query methods
- Custom JPQL queries
- Native SQL queries
- Pagination and sorting
- Entity relationships and joins
- Constraints and validations
- Transaction boundaries

EXAMPLE TEST

```
@DataJpaTest
class EmployeeRepositoryTest {
  @Autowired
  private EmployeeRepository employeeRepository;

  @Test
  void should_find_employee_by_email() {
     Employee emp = new Employee("John Doe", "john.doe@example.com",
                    "Engineering");
    employeeRepository.save(emp);

    Optional=Employee> found = employeeRepository.findByEmail("john.doe@example.com");

    assertTrue(found.isPresent());
    assertEquals("John Doe", found.get().getName());
    assertEquals("Engineering", found.get().getDepartment());
}
```

TEST DATA STRATEGY

Insert
Use @BeforeEach or test data builders to set up data.

Rollback
@DataJpaTest rolls back each test to keep data isolated.

Testcontainers (Optional)
Use PostgreSQL Testcontainers for environment parity.

BEST PRACTICES

Test one repository behavior per test
Keep tests small and focused.

Use meaningful test data
Use realistic data to validate queries.

Prefer derived queries
Use custom queries only when needed.

Keep tests fast and isolated
Avoid loading full application context.

Verify edge cases and constraints
Test nulls, duplicates, and boundary values.

COMMON ANNOTATIONS

@DataJpaTest	Configures JPA slice for repository tests.
@Autowired	Injects repository into the test.
@Test	Marks test methods.
@Transactional	Ensures rollback after each test.
@Sql	Execute SQL scripts before/after tests.

KEY TAKEAWAY: Repository integration tests ensure your data access layer is correct, reliable, and performs as expected with a real database.

spring boot

SERVICE INTEGRATION TESTING

A partial context wires the real service to a real repository, while unrelated collaborators stay mocked.

Real Service + Real Repository

Unlike a pure unit test, the service under test talks to an actual repository backed by a real (or Testcontainers) database.

Unrelated Collaborators Stay Mocked

An event publisher the service calls can still be a test double -- only the boundary under test needs to be real.

Proves Transactional Behavior

A service integration test is where @Transactional boundaries and rollback-on-exception behavior actually get exercised.

KEY TAKEAWAY Service integration tests sit deliberately between a pure unit test (everything mocked) and a full HTTP test (everything real) -- they prove the service's own logic against a genuinely real persistence boundary.

TRY IT YOURSELF — EXERCISE 2: PLAN THE SPRING INTEGRATION TESTS

Reinforces: @SpringBootTest, test slices, and controller, repository, and service integration testing as you just covered them.

STEPS (notes/lab19-spring-test-plan.md)

Step	What To Do
1 — Slice Choice	For each test, pick the narrowest annotation that still proves what you need.
2 — Two Cases	Write one GET case for CUS-1001 and one POST case for a new customer.
3 — What It Proves	Say what an integration test proves that a unit test cannot.
4 — Context Cost	Say why loading the full context in every test slows the suite.

Slice selection

```
@WebMvcTest      controller mapping + serialisation, service mocked
@DataJpaTest     repository queries against a real database
@SpringBootTest  full wiring -- use sparingly, it is the slowest
```

TRY IT NOW — Complete Exercise 2 before continuing: exercises/exercise-02-spring-test-plan.md (workspace: examples/module-19-exercises).

SERVICE INTEGRATION TESTING

Service integration testing validates the business logic in the service layer in combination with its dependencies (repositories, external services, mappers, etc.).

WHY SERVICE INTEGRATION TESTING?

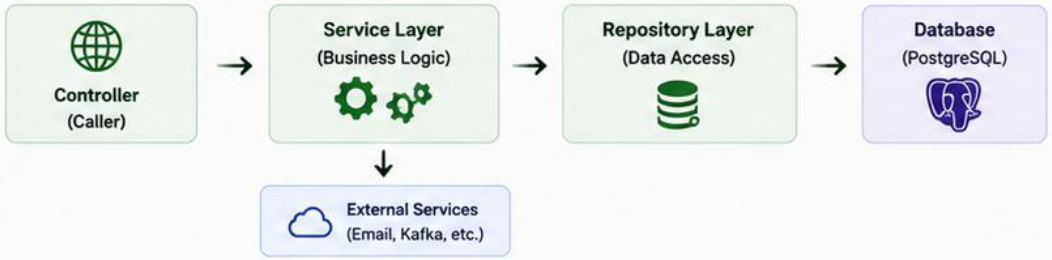
Validate Business Logic
Ensures service methods implement rules, workflows, and validations correctly.

Test Collaborations
Verifies interactions with repositories and external dependencies.

Higher Confidence
Catches integration issues between layers early.

Realistic Execution
Runs with real context configuration and transactions.

SERVICE LAYER CONTEXT



TYPICAL TEST FLOW

1. Arrange Test Data (Set up mocks / test data)
2. Invoke Service Method (With input parameters)
3. Verify Interactions (Repositories, External Services)
4. Assert Results (Return values, state changes)

WHAT WE TEST

- Service method business logic
- Data validations and rules
- Interactions with repositories
- External service integrations
- Exception handling
- Transaction boundaries
- Edge cases and error scenarios

EXAMPLE TEST (JUnit 5 + Mockito)

```
@SpringBootTest
class EmployeeServiceIntegrationTest {

    @Autowired
    private EmployeeService employeeService;

    @MockBean
    private EmployeeRepository employeeRepository;

    @MockBean
    private EmailService emailService;

    @Test
    void should_create_employee_successfully() {
        Employee emp = new Employee(null, "John Doe", "john.doe@example.com");
        when(employeeRepository.save(any(Employee.class)))
            .thenAnswer(inv -> {
                Employee e = inv.getArgument(0);
                e.setId(1L);
                return e;
            });

        EmployeeDTO result = employeeService.createEmployee(emp);

        assertNotNull(result.getId());
        assertEquals("John Doe", result.getName());
        assertEquals("john.doe@example.com", result.getEmail());

        // Verify interactions
        verify(employeeRepository).save(any(Employee.class));
        verify(emailService).sendWelcomeEmail(anyString());
        verifyNoMoreInteractions(employeeRepository, emailService);
    }
```

ANNOTATIONS USED

@SpringBootTest	Loads full Spring context.
@MockBean	Creates and injects mock beans.
@Autowired	Injects real service bean.
@Transactional	Ensures rollback after test.
@Test	Marks a test method.

BEST PRACTICES

- Keep tests focused on one service behavior per test.
- Mock external systems, test them separately.
- Use meaningful test data and builders.
- Verify interactions and outcomes.
- Keep tests fast, reliable, and isolated.

COMMON PITFALLS

- Testing multiple behaviors in one test.
- Over-mocking external details.
- Ignoring exception scenarios.
- Not verifying interactions.
- Using real external services in unit tests.

KEY TAKEAWAY: Service integration tests ensure your service layer logic works correctly with its dependencies, providing confidence in business functionality.

spring boot

TEST DATABASE STRATEGY

H2 is fast but not PostgreSQL -- choose deliberately, per test, rather than defaulting to whatever's convenient.

STRATEGY	WHEN IT'S THE RIGHT CHOICE
H2 in-memory	Fast, basic slice tests where SQL-dialect compatibility genuinely doesn't matter.
Testcontainers PostgreSQL	Any repository or service test where real query behavior, constraints, or functions matter.
Shared test PostgreSQL instance	CI environments where spinning up a fresh container per run is costly -- reset schema between suites.
Never: production database	A test must never run against production data or infrastructure, ever.
Rule of thumb	If @DataJpaTest's default embedded DB is left unconfigured, know that's an explicit choice, not a neutral default.

KEY TAKEAWAY This course's PostgreSQL-backed test suite uses Testcontainers wherever query correctness genuinely matters -- H2 is reserved for the narrow cases where it's provably safe.

TEST DATABASE STRATEGY

A solid test database strategy ensures fast, reliable, and isolated tests that accurately validate data access and business logic.

WHY IT MATTERS

Reliable Tests
Consistent data and environment produce dependable results.

Fast Execution
Optimized test databases reduce test time and increase feedback.

Isolation
Tests do not impact development or production data.

Repeatability
Database state is reset between tests for consistent outcomes.

Cost Effective
Lightweight, in-memory or container databases reduce infrastructure costs.

STRATEGY OVERVIEW

1. **Choose Right Database** — Select the right database for the test type.
2. **Provision Test Environment** — Use automation (containers, scripts) to provision the database.
3. **Prepare Schema & Data** — Apply migrations and load test data consistently.
4. **Isolate & Reset Between Tests** — Clean state before or after each test execution.
5. **Optimize Performance** — Use lightweight options and tune for speed.

SUPPORTED OPTIONS

H2 Database
In-memory, fast, ideal for unit & slice tests.

PostgreSQL
Real PostgreSQL instance (local or container).

Testcontainers
Spin up real DB in Docker for integration tests.

Embedded DB (HSQLDB/Derby)
Lightweight, in-process option.

APPROACH COMPARISON

Approach	Pros	Cons	Best For
H2	• Very fast • Easy to setup • Zero external dependency	• Not 100% compatible with PostgreSQL • Limited features	Unit tests, repository slices
	• Production-like • Full feature set • Accurate SQL behavior	• Slower than H2 • Requires DB instance	Integration tests
	• Real DB in Docker • Isolated • Reproducible	• Requires Docker • Slightly slower startup	Integration tests (CI/CD friendly)
	• Very lightweight • No external services	• Fewer features • Deprecated in some cases	Legacy tests

TEST DATABASE LIFECYCLE (Example with Flyway)

1. **Provision** — Create database (container/script).
2. **Migrate** — Run Flyway/Liquibase migrations.
3. **Seed Data** — Load reference or test data.
4. **Execute Tests** — Run tests using this database.
5. **Clean Up** — Truncate or drop schema/db.

TEST DATA STRATEGY

Deterministic
Use known, predictable data sets.

Minimal
Load only the data required for the test.

Reusable
Use builders, fixtures, or SQL scripts.

Independent
Each test should be independent of others.

BEST PRACTICES

- Automate database setup and cleanup.
- Keep tests independent and isolated.
- Run migrations in tests to match production schema.
- Use transactions or clean state between tests.
- Use Testcontainers in CI for production-like tests.

KEY TAKEAWAY: A well-designed test database strategy ensures accuracy, speed, and reliability across your test suite.

spring boot

POSTGRESQL TEST ENVIRONMENT

A disposable, real PostgreSQL container gives every test run the same clean, production-accurate starting point.

Container Lifecycle

@Container manages the PostgreSQLContainer's start/stop -- typically one instance shared across a test class for speed.

Dynamic Wiring

@DynamicPropertySource injects the running container's actual JDBC URL, username, and password into the Spring context.

Clean Slate Per Run

A freshly started container has no leftover data -- schema is created fresh via migrations or Hibernate DDL each run.

KEY TAKEAWAY The same PostgreSQL major version this course's application uses in production is exactly what the test container should run -- version drift between test and production defeats the whole point.

PostgreSQL Test Environment

A dedicated, isolated PostgreSQL environment ensures reliable, repeatable, and fast integration tests.

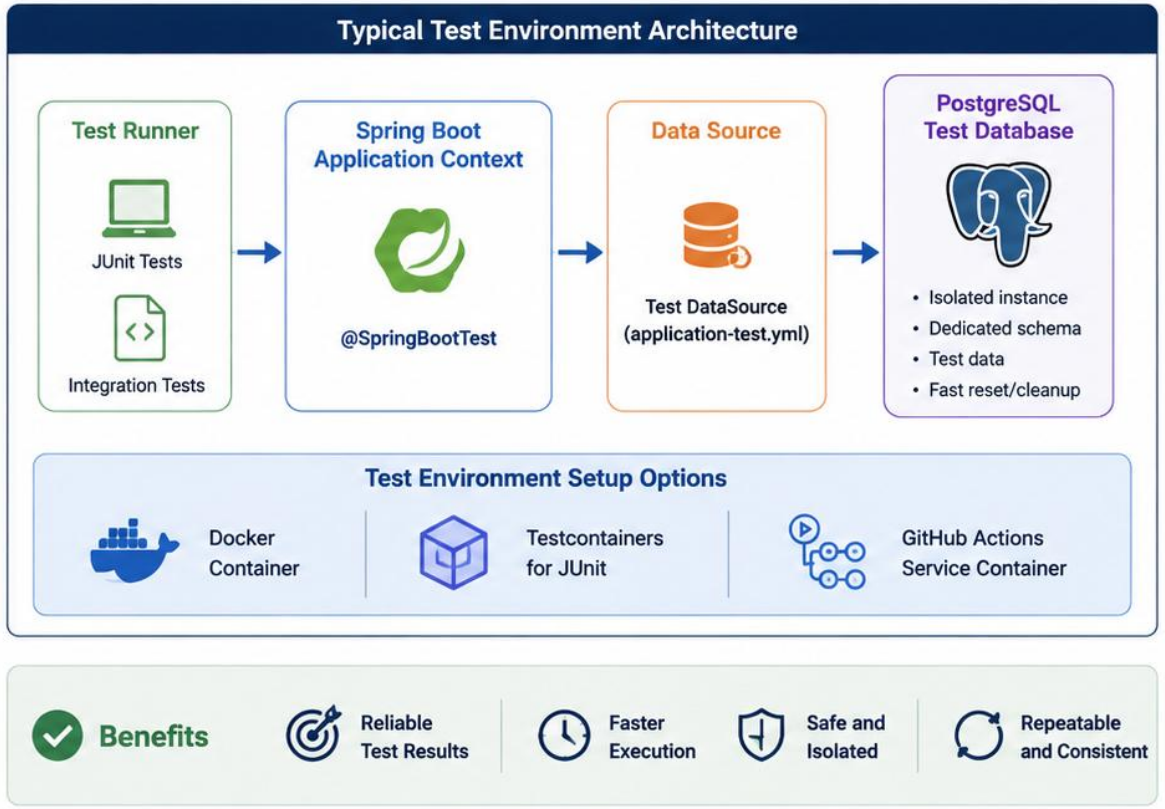
Isolated from Development and Production
Tests run against a separate PostgreSQL instance to ensure no impact on real data.

Fast and Repeatable
Database is created fresh or cleaned between test runs to maintain consistency.

Secure and Controlled
Limited access, minimal permissions, and network isolation protect the test environment.

Automated Provisioning
Use Docker/Testcontainers or CI pipelines to spin up the test database automatically.

Realistic and Lightweight
Mirrors production schema and settings while remaining lightweight for local and CI execution.



Best Practice: Keep your test database close to production in schema and configuration, but never share data or environments.

Clean data. Fast tests. Confident delivery.

TEST DATA MANAGEMENT

Deterministic, seeded test data is what makes an integration test's assertions actually meaningful.

Seed Before Each Test

Insert known, fixed rows (e.g. CUS-1001) before the test runs, rather than relying on leftover data from a previous run.

Fixed, Recognizable IDs

Consistent fixture IDs across the whole test suite make failures easy to trace back to a specific, known scenario.

Clean Up After

Whatever a test inserts, it should remove or roll back -- so the next test starts from the same known state.

KEY TAKEAWAY An assertion like 'expect exactly 3 customers' is only meaningful if the test controls exactly what data existed before it ran -- never rely on 'whatever happens to be in the database.'

Test Data Management

Well-managed test data ensures accurate, reliable, and repeatable integration and UI test results.

Consistent and Reliable
Use predictable datasets so tests produce the same results every time.

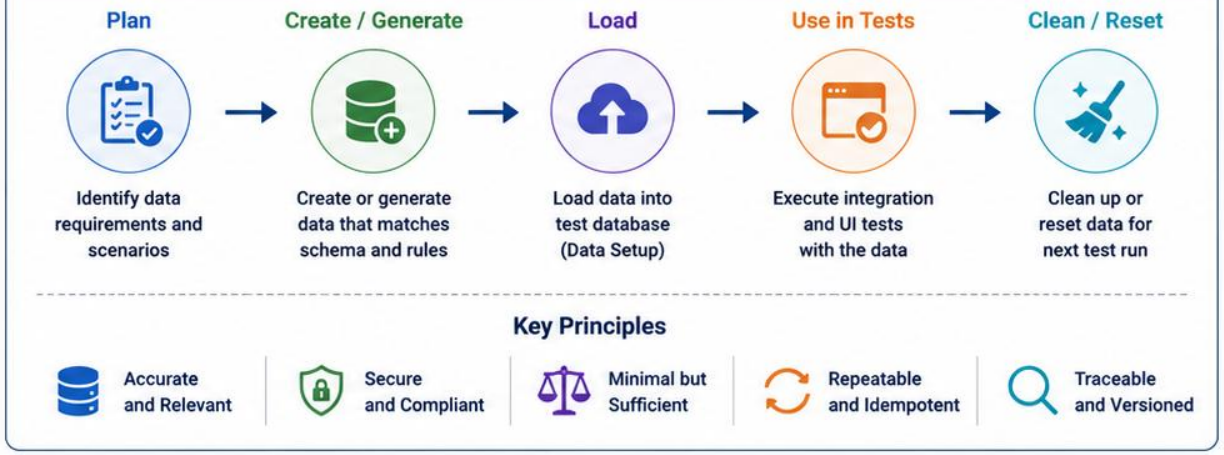
Isolated and Safe
Test data is separated from real data to avoid impact on development or production.

Versioned and Maintainable
Track changes to datasets over time for traceability and repeatability.

Automated and Reusable
Load and clean data automatically to save time and reduce manual effort.

Representative and Complete
Cover key scenarios, edge cases, and business rules with realistic test data.

Test Data Lifecycle



Key Principles

- Accurate and Relevant
- Secure and Compliant
- Minimal but Sufficient
- Repeatable and Idempotent
- Traceable and Versioned

Test Data Management Approaches

- **Static Data** — Fixed datasets for stable scenarios
- **Dynamic Data** — Generated data for variable scenarios
- **Template-Based** — Parameterized templates for flexible datasets
- **Snapshot / Clone** — Database snapshots for environment reset

Best Practices

- Keep data minimal but meaningful
- Avoid hard-coding data in tests
- Use factories and builders where possible
- Mask sensitive data (PII, secrets)
- Automate setup and cleanup in CI/CD

Best Practice: Good test data is the foundation of reliable tests.

Clean data. Accurate tests. Confident delivery.

TEST ISOLATION

@Transactional rollback isolates test data cleanly -- but read the fine print before relying on it everywhere.

@Transactional Rollback

Wrapping a test in a transaction that's rolled back after it finishes gives each test a clean slate, automatically.

Can Hide Commit-Time Behavior

A rollback-based test never truly commits -- it can miss bugs that only appear once data is genuinely persisted.

Use Explicit Commits When It Matters

When behavior genuinely depends on a real commit, a non-transactional test is the correct, deliberate choice.

KEY TAKEAWAY Read the fine print: @DataJpaTest's embedded database and @Transactional's rollback can hide real production behavior unless you configure them deliberately.

TRY IT YOURSELF — EXERCISE 3: CHOOSE THE TEST DATABASE STRATEGY

Reinforces: the test database strategy, PostgreSQL test environment, data management, and isolation slides you just covered.

STEPS (notes/lab19-postgres-test-strategy.md)

Step	What To Do
1 — Pick One	Choose Testcontainers, a dedicated test database, or an in-memory database, and say why.
2 — Same Engine	Say what breaks when tests use H2 but production uses PostgreSQL.
3 — Isolation	Say how each test starts from a known state, and name the mechanism.
4 — Seed Data	Say where Amina and Ravi come from and that no real data is used.

Test database choice

```
Testcontainers: real PostgreSQL per run -- same engine, disposable
isolation: @Transactional rollback, or truncate between tests
seeds: CUS-1001 Amina, CUS-1002 Ravi -- synthetic only, never real
```

TRY IT NOW — Complete Exercise 3 before continuing: exercises/exercise-03-test-db-strategy.md (workspace: examples/module-19-exercises).

Test Isolation

Isolating tests ensures that each test runs independently, produces reliable results, and does not affect other tests.

Independent Execution
Tests should run in any order, any time, and produce the same result.

Isolated Data
Each test uses its own data that is not shared or modified by other tests.

Isolated Environment
Run tests in separate environments or containers to avoid external dependencies.

Clean State Between Tests
Reset data and environment after each test to ensure a clean starting point.

Predictable and Reliable
Isolation eliminates side effects, reduces flakiness, and builds confidence in test results.

Isolation Layers

Test Runner	Application Context	Test Database	External Services	File System / Cache
Runs tests independently	Each test loads its own application context	Separate database or schema per test or test class	Mock or stub external services (isolated)	Use temp directories and clear caches between tests

Benefits of Test Isolation

- Reliable Results
- Reduced Flakiness
- Faster Debugging and Maintenance
- Safe Parallel Execution
- Greater Confidence in Releases

Common Isolation Techniques

Database Per Test	Transaction Rollback	Data Cleanup	Containerized Environments	Mock External Dependencies
Create a new database or schema for each test or test class.	Wrap tests in a transaction and roll back after completion.	Delete test data after each test execution.	Use Docker/Testcontainers to spin up isolated environments.	Use mocks/stubs to isolate tests from external systems.

Best Practice: Design tests to be independent, use isolated data sources, and clean up after execution.

★ Isolated tests today, reliable applications tomorrow.

INTRODUCTION TO SELENIUM

Selenium WebDriver is the widely adopted standard for driving a real browser from test code.

Cross-Browser Standard

Selenium WebDriver implements the W3C WebDriver protocol, working consistently across Chrome, Firefox, and Edge.

Drives a Real Browser

Unlike a component test, Selenium controls an actual browser rendering the actual compiled Angular application.

Reserved for Critical Journeys

Per the test pyramid, Selenium tests are few and slow -- used for the Northstar CRM's most critical user flows only.

KEY TAKEAWAY Selenium WebDriver is the modern standard: IDE-free, W3C-standardized, and language-agnostic -- Java bindings drive the exact same protocol as every other Selenium client.

Introduction to Selenium

Selenium is an open-source suite of tools used for automating web browsers to test web applications across platforms.

Open Source
Selenium is a widely adopted open-source framework supported by a large community.

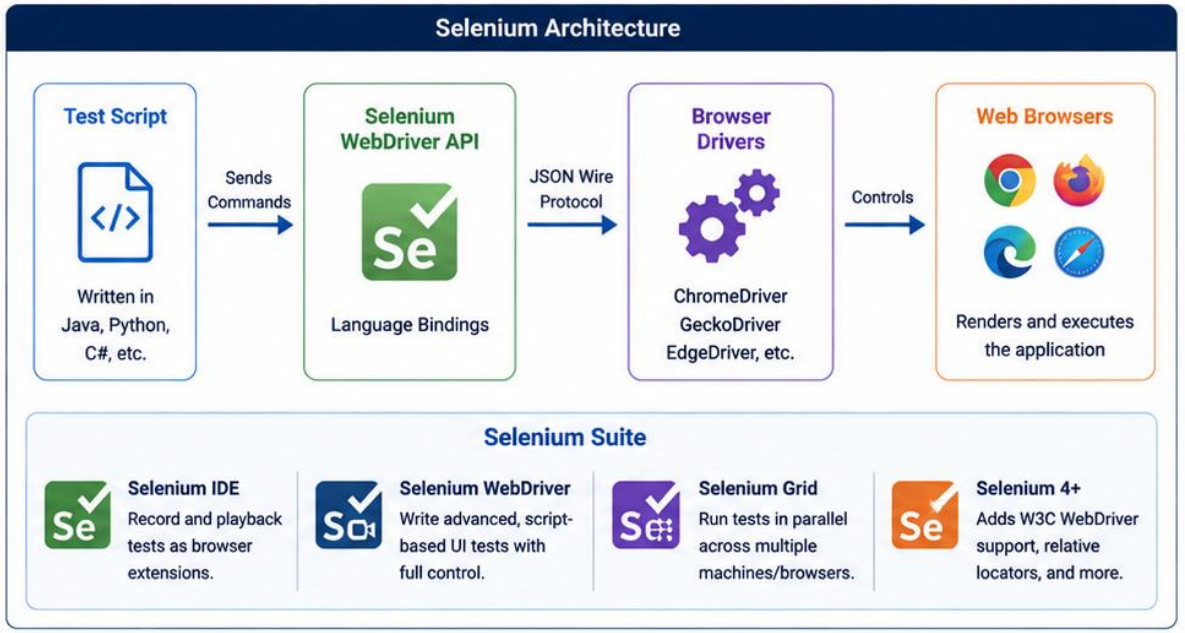
Cross-Browser
Execute tests on all major browsers including Chrome, Firefox, Edge, and Safari.

Cross-Platform
Run tests on Windows, macOS, and Linux environments.

Multi-Language Support
Write tests in Java, Python, C#, JavaScript, Ruby, Kotlin and more.

Automation at Scale
Integrates with CI/CD tools to enable reliable, repeatable test automation.

Selenium Architecture



Key Capabilities

Best Practice: Follow the Page Object Model, use stable selectors, and wait for elements properly to build reliable and maintainable UI tests.

★ Test like a user. Automate with confidence.

SELENIUM ARCHITECTURE

Test code, a driver process, and a real browser -- three pieces talking over a standardized wire protocol.

1 Test Script

Java test code calls Selenium's WebDriver API -- driver.get(), findElement(), click(), and so on.

2 Browser Driver

chromedriver translates WebDriver protocol commands into browser-native automation calls.

3 Real Browser

Chrome actually loads and renders the Angular app, executes its JavaScript, and reports back what's on screen.

KEY TAKEAWAY Selenium follows a client-server architecture: test scripts talk to a browser driver over HTTP, and the driver talks to the real browser.

Selenium Architecture

Selenium follows a client-server model where test scripts communicate with browser drivers to automate browsers.

Client-Server Model
Test scripts (clients) send commands to the Selenium WebDriver, which communicates with browser drivers.

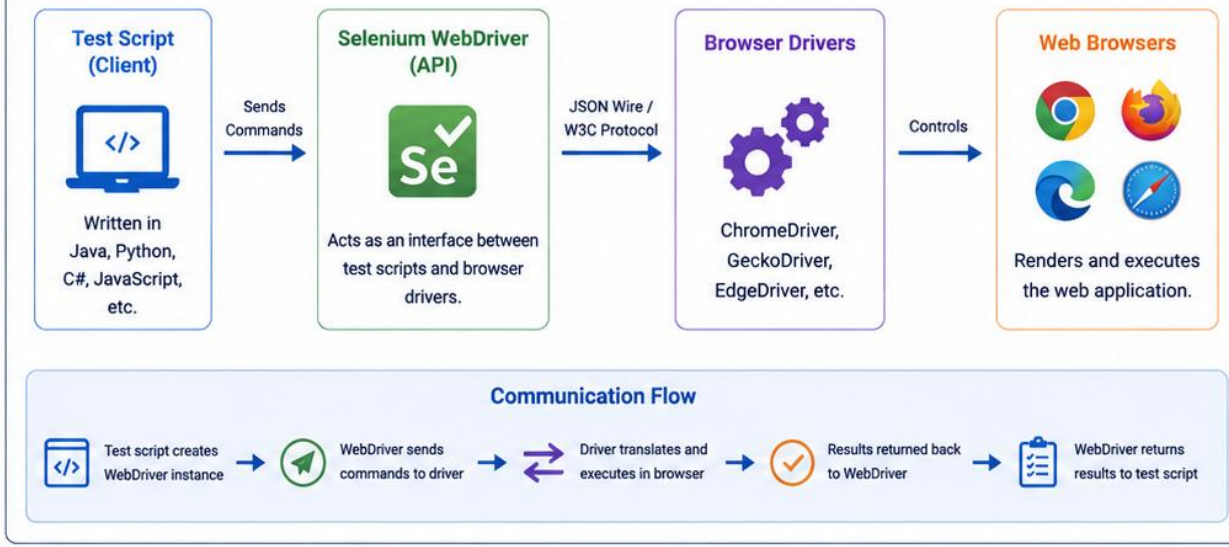
Language Bindings
Selenium provides bindings for multiple programming languages to write test scripts.

WebDriver Protocol
WebDriver uses the W3C WebDriver protocol to send commands and receive responses in JSON format.

Browser Drivers
Browser-specific drivers (ChromeDriver, GeckoDriver, etc.) act as a bridge between WebDriver and browsers.

Real Browsers
Drivers control real browsers to render the application and execute user interactions.

Selenium Architecture Overview



Key Selenium Components

Selenium IDE
Record and playback tests as browser extensions.

Selenium WebDriver
Write robust, script-based functional tests.

Selenium Grid
Run tests in parallel across multiple machines/browsers.

Selenium 4+
W3C WebDriver support, relative locators, CDP, and more.

Best Practice: Keep browser drivers updated and version-matched with the browsers for stable test execution.

★ Well-structured architecture leads to maintainable, scalable, and reliable UI tests.

WEBDRIVER FUNDAMENTALS

A small, consistent API -- navigate, locate, interact, inspect, and always clean up.

Navigate

driver.get("http://localhost:4200/customers") loads a URL, just like typing it into the address bar.

Locate & Interact

driver.findElement(By...) locates one element; .click(), .sendKeys(), and .getText() interact with or read it.

Always Quit

driver.quit() in an @AfterEach ensures the browser process is closed, even when a test fails.

KEY TAKEAWAY WebDriver is the API you use to control the browser -- a small, learnable vocabulary that covers nearly every UI interaction this module needs.

WebDriver Fundamentals

WebDriver is the core interface in Selenium that provides methods and controls to automate web browsers.

Browser Automation
WebDriver controls real browsers just like a real user, enabling functional and UI testing.

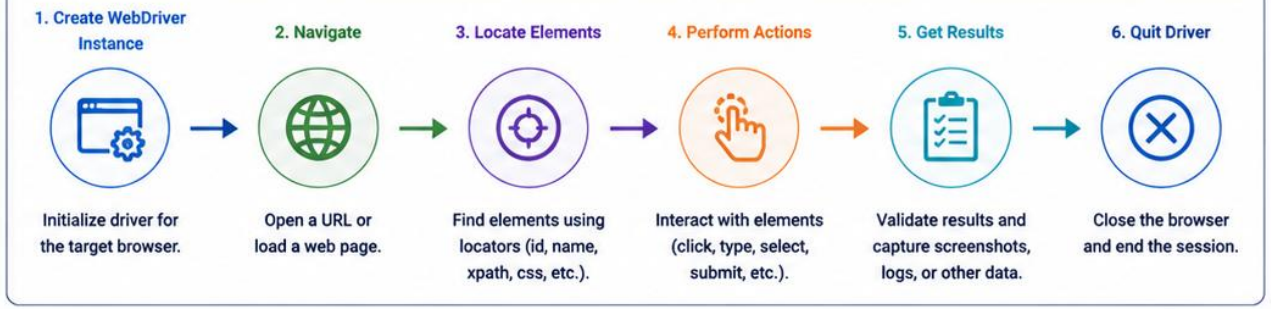
Simple and Consistent API
Provides a consistent set of methods to locate elements, perform actions, and navigate pages.

Language Bindings
Implements the same API across multiple languages like Java, Python, C#, JavaScript, etc.

Browser Independence
Supports all major browsers through dedicated drivers (ChromeDriver, GeckoDriver, etc.).

Cross-Platform
Run tests on Windows, macOS, Linux in local or remote environments.

WebDriver Workflow



Common WebDriver Actions

- **navigate().to(url)** — Open a URL
- **getText()** — Get text from an element
- **findElement(By locator)** — Locate a single element
- **getAttribute(name)** — Get attribute value
- **click()** — Click on an element
- **navigate().back()** — Navigate back
- **sendKeys(text)** — Type text into an element
- **getScreenshotAs()** — Capture screenshot

WebDriver Interface (Java)

```java
WebDriver driver = new ChromeDriver();
driver.manage().window().maximize();
driver.get("https://example.com");

WebElement username = driver.findElement(By.id("user"));
username.sendKeys("john");

WebElement loginBtn = driver.findElement(By.cssSelector("button.login"));
loginBtn.click();

// Validate
String title = driver.getTitle();
System.out.println("Page Title: " + title);

driver.quit();
```

Best Practice: Always quit the driver after test execution, use explicit waits, and keep locators stable and maintainable.

Key Takeaway: WebDriver provides the foundation for reliable and powerful browser automation in Selenium.

BROWSER AUTOMATION WORKFLOW

Every Selenium test, however different the scenario, follows the same five-step shape.

1 Launch & Navigate

Start the browser, navigate to the Angular app's URL, and wait for it to be ready for interaction.

2 Locate & Act

Find the relevant elements with a stable selector, then click, type, or select as the scenario requires.

3 Wait, Assert, Quit

Explicitly wait for the resulting UI state, assert what's on screen, then always quit the driver.

KEY TAKEAWAY This five-step shape -- launch, navigate, locate, act, wait-assert-quit -- is the mental checklist behind every Selenium test this module writes, from a login flow to a form submission.

Browser Automation Workflow

Selenium automates real user interactions in a browser by following a structured workflow.

Define Test Scenario
Identify the user flow and define the steps to automate.

Write Test Script
Implement the scenario using Selenium WebDriver APIs.

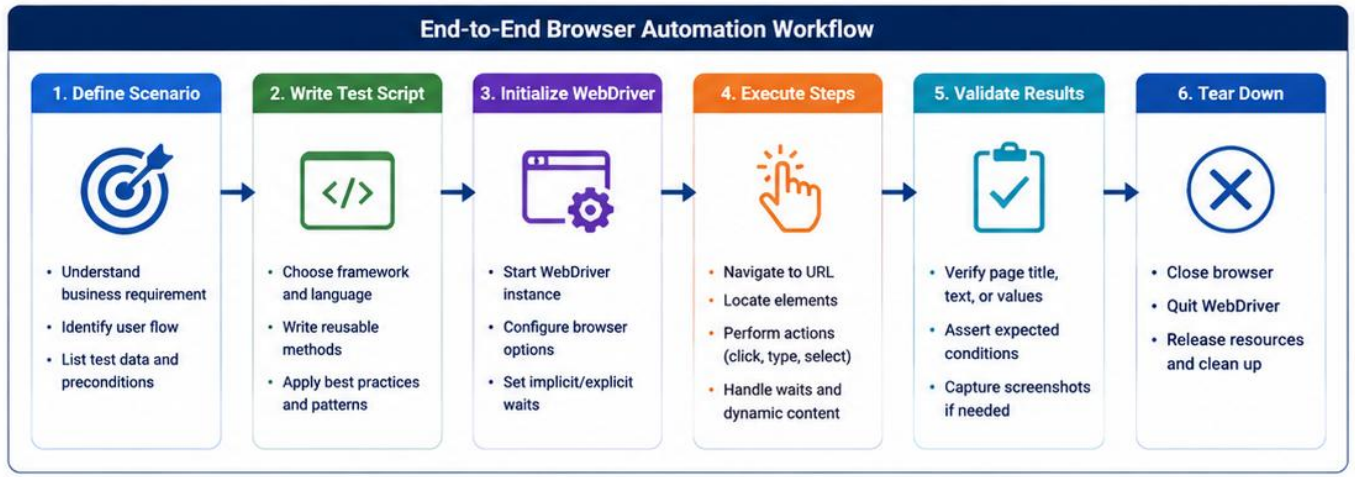
Initialize WebDriver
Create a WebDriver instance for the target browser.

Execute Steps
Perform actions like click, type, select, navigate, and wait.

Validate Results
Verify expected outcomes using assertions and validations.

Tear Down
Close the browser and release resources after test execution.

End-to-End Browser Automation Workflow



Key WebDriver Actions

navigate().to(url)	Open a URL
findElement(locator)	Locate a single element
click()	Click on an element
sendKeys(text)	Enter text into an element
getText()	Get visible text
getScreenshotAs()	Capture a screenshot

Common Wait Strategies

Implicit Wait
Set a global wait time for all element searches.

Explicit Wait
Wait for a specific condition before proceeding.

Fluent Wait
Poll for a condition at regular intervals within a timeout.

Best Practices

- Use meaningful and stable locators
- Avoid hard waits; use explicit waits
- Follow the Page Object Model
- Keep tests independent and idempotent
- Handle exceptions and take screenshots
- Close the browser in teardown

Best Practice: Automate like a user, validate like a test engineer, and clean up like a professional.

Key Takeaway: A structured workflow ensures reliable, maintainable, and repeatable browser automation.

ANGULAR APPLICATION TESTING WITH SELENIUM

An Angular SPA behaves differently from a classic multi-page site -- Selenium tests must account for that.

Client-Side Routing

The Angular Router changes views without a full page reload -- driver.get() is only used once, at the start.

Asynchronous by Default

Nearly every meaningful UI change follows an HTTP call to Spring Boot -- content isn't there the instant an action fires.

Component-Driven DOM

The DOM is generated by Angular components at runtime, not static HTML -- elements appear and vanish as *ngIf/*ngFor evaluate.

KEY TAKEAWAY Testing a single-page application means testing a moving target -- the DOM Selenium queries a moment ago may already be different by the time an assertion runs.

Angular Application Testing with Selenium

Selenium can be effectively used to test Angular applications by understanding how Angular renders and updates the DOM.

Test Real User Workflows
Automate end-to-end user journeys across pages, components, and features.

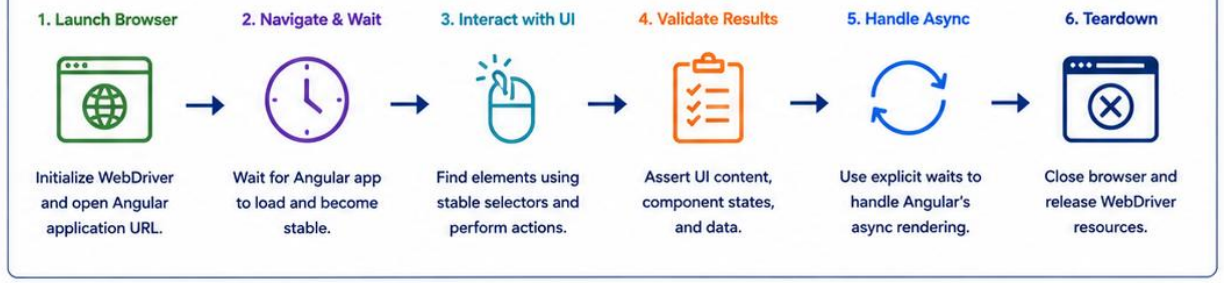
Handle Angular-Specific Behavior
Account for dynamic rendering, routing, data binding, and asynchronous updates.

Stable Element Identification
Use reliable selectors and wait strategies to interact with Angular components.

Validate UI and Data
Verify UI states, forms, API-driven data, and user interactions.

Cross-Browser and Headless Execution
Run tests across browsers locally or in CI/CD pipelines.

Angular + Selenium Test Flow



Angular Application Characteristics
- Single Page Application (SPA) with client-side routing
- Dynamic DOM updates and component re-rendering
- Data binding and asynchronous API calls
- Loading indicators and spinners
- Component-based UI structure

Important Testing Considerations
- Wait for Angular to stabilize before interacting
- Avoid brittle selectors tied to CSS classes
- Prefer data-testid or data-qa attributes
- Handle route changes and lazy loaded modules
- Mock or seed data for predictable tests

Common Use Cases
- Login and authentication flows
- Form submissions and validations
- CRUD operations on data grids
- Navigation and routing tests
- Error handling and empty states

Best Practice: Use stable selectors, apply explicit waits, and synchronize tests with Angular's rendering lifecycle.

Tech Stack: Selenium WebDriver, TypeScript/Java/JavaScript, TestNG/JUnit, Angular, ChromeDriver/GeckoDriver

Key Takeaway: Understanding Angular's behavior and using robust Selenium strategies leads to reliable and maintainable UI tests.

UNDERSTANDING ANGULAR DOM RENDERING

Angular's change detection re-renders the DOM whenever component state changes -- Selenium sees the result, not the process.

Change Detection Cycles

Angular re-evaluates bindings and updates the DOM whenever state changes -- typically after an event or an Observable emission.

Elements That Don't Exist Yet

A *ngIf-gated element isn't in the DOM until its condition becomes true -- findElement() throws, it doesn't just find nothing.

Repeated Elements from *ngFor

A customer table's rows are generated per array item -- the row count in the DOM should exactly match the seeded fixture data.

KEY TAKEAWAY A NoSuchElementException from Selenium against an Angular app is very often not a bug in the app -- it's a test that queried the DOM before Angular finished rendering the relevant state.

Understanding Angular DOM Rendering

Angular builds the DOM dynamically based on your component templates, state, and data.

Key Points

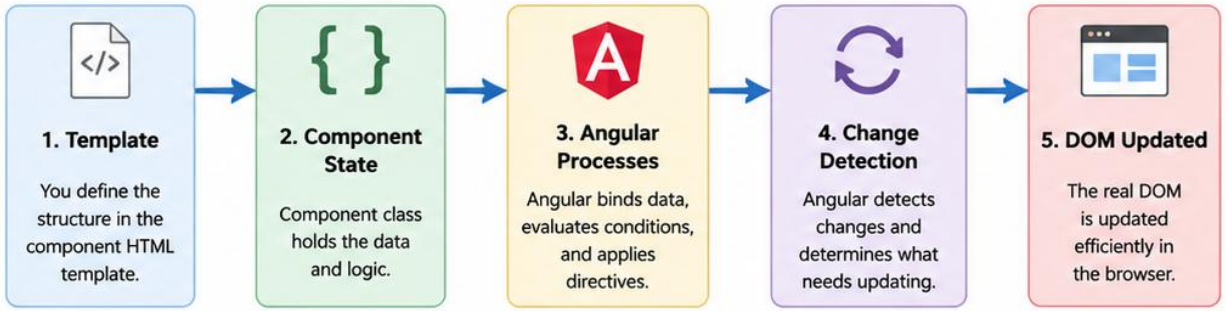
- **Template-Driven UI:** Angular templates describe what the UI should look like.
- **Data Binding:** Component data is bound to the template using interpolation, property binding, and directives.
- **Change Detection:** Angular detects data changes and updates the DOM efficiently.
- **Asynchronous Rendering:** Data from APIs, events, or user interactions can change the DOM after initial load.
- **Virtual DOM Concept:** Angular uses a change detection mechanism (not a real virtual DOM) to minimize DOM updates.

Why It Matters

Understanding how Angular renders the DOM helps you:

- Choose stable selectors for UI testing
- Handle timing issues in tests
- Avoid flaky tests caused by dynamic updates
- Write more reliable, maintainable automation scripts

How Angular Renders the DOM



Example

Component Template (employee-list.component.html)

```
<div class="card">
  <h2>Employees</h2>
  <ul>
    <li *ngFor="let emp of employees">
      <span>{{ emp.name }}</span> - {{ emp.department }}
    </li>
  </ul>
</div>
```

Angular renders the `<li>` elements dynamically based on the `employees` array.

Rendered DOM in Browser

Employees

- John Doe - Engineering
- Jane Smith - Finance
- Robert Brown - HR

Key Takeaway: Angular renders the DOM dynamically based on your data and interactions. Understanding this process is critical for writing stable UI tests and building robust applications.

Stable Tests Start with Understanding the DOM

CHOOSING STABLE SELECTORS

A selector should survive a CSS refactor -- if it breaks when a designer changes a class name, it was the wrong selector.

data-testid First

A dedicated attribute like data-testid="submit-customer" exists purely for tests -- it never changes for styling reasons.

Not Generated CSS Classes

Angular Material and utility CSS frameworks generate class names that can change between versions or builds.

Not Visible Text Alone

Button text is a localization and copy-editing target -- a selector tied to exact text breaks the moment a string changes.

KEY TAKEAWAY Selenium WebDriver is the modern standard: ID-based and stable data attributes come first, generated CSS or brittle text-matching selectors come last.

CHOOSING STABLE SELECTORS

Use selectors that are reliable, predictable, and resistant to UI changes.

BEST PRACTICES

Use Semantic Attributes

Prefer id, name, role, aria-label, or data-* attributes.

Use data-testid / data-qa

Add custom attributes for testing that are not affected by UI changes.

Avoid Presentation-Based Selectors

Do not rely on CSS classes, positions, or styling.

Be Specific but Not Fragile

Create selectors that uniquely identify the element without depending on its structure.

Resilient to UI Changes

Choose selectors that remain stable even if the layout, themes, or styling change.

EXAMPLES: STABLE vs FRAGILE SELECTORS

	PREFERRED (STABLE)	AVOID (FRAGILE)
Username 	By ID `#username`	By CSS Class `.form-control`
	By data-testid `[data-testid="submit-btn"]`	By Tag / Position `button:nth-child(2)`
Status 	By Name / Role `[name="status"]`	By Deep CSS `div.container > div > select`
	By aria-label `[aria-label="Delete item"]`	By Icon Class `.icon-delete`
View Details	By Link Text / Role `a[role="link"][href*="details"]`	By Absolute XPath `/html/body/div[1]/div[3]/a`

RECOMMENDATION

Add dedicated test attributes (e.g., data-testid, data-qa) in Angular templates. This makes your tests more stable, readable, and maintainable.

```
<button
  data-testid="submit-btn"
  class="btn btn-primary">Submit</button>
```

Key Takeaway: Stable selectors make your UI tests reliable and resistant to UI changes, leading to fewer flaky tests and lower maintenance.

ID AND ATTRIBUTE SELECTORS

A ranked list of locator strategies, from most to least stable, for this course's Angular application.

LOCATOR STRATEGY	EXAMPLE
By.cssSelector on data-testid	By.cssSelector("[data-testid='customer-email']")
By.id	By.id("customerEmail") -- stable if the Angular template sets a genuinely fixed id.
By.name	By.name("email") -- useful for form controls with a stable name attribute.
By.cssSelector (structural, minimal)	By.cssSelector("form[data-testid='customer-form'] input[type='email']")
Avoid: By.xpath tied to text/position	//div[3]/span[text()='Save'] -- breaks on any layout or copy change.

KEY TAKEAWAY Rank locator strategies the same way every time: dedicated test attributes first, genuinely stable IDs and names next, minimal structural CSS after that, and brittle position- or text-based XPath as an absolute last resort.

ID AND ATTRIBUTE SELECTORS

Use IDs and attributes to locate elements reliably and make your UI tests more stable.

ID SELECTORS

IDs are unique within the page and are the most reliable selectors.

```
<input id="username" type="text" placeholder="Enter username">
<button id="submitBtn" class="btn btn-primary">Submit</button>
```

How to use

```
#username          // Selects the input with id="username"
#submitBtn         // Selects the button with id="submitBtn"
```

ATTRIBUTE SELECTORS

Attributes help select elements when IDs are not available.

```
<input type="text" name="email" placeholder="Enter email">
<button data-test="save-user" class="btn btn-success">Save</button>
```

How to use

```
[name="email"]          // Selects element with name="email"
[data-test="save-user"] // Selects element with data-test="save-user"
[type="text"]           // Selects all elements with type="text"
[placeholder="Enter email"] // Selects by placeholder attribute
[aria-label="Close dialog"] // Selects by aria-label attribute
```

EXAMPLES IN AN ANGULAR TEMPLATE

UI Element	HTML Example	Recommended Selector
Username Input — Enter username	`<input id="username" type="text">`	`#username`
Email Input — Enter email	`<input name="email" type="email">`	`[name="email"]`
Save Button — Save	`<button data-test="save-user">Save</button>`	`[data-test="save-user"]`
Status Dropdown — Active	`<select aria-label="Status">.../select>`	`[aria-label="Status"]`

BEST PRACTICES

- Prefer **ID selectors (#id)** when available.
- Use custom attributes like data-test or data-qa for testing.
- Avoid using dynamic attributes or values that may change.
- Do not rely on CSS classes for tests (they are prone to UI changes).
- Make selectors meaningful and easy to understand.

GOOD	AVOID
`#submitBtn`	`.btn.btn-primary`
`[data-test="search"]`	`button:nth-child(2)`
`[aria-label="Close"]`	`div > input[type="text"]`

Key Takeaway: ID and attribute selectors are stable, predictable, and resistant to UI changes.
Use them to write reliable and maintainable UI automation tests.

AVOIDING FRAGILE CSS / XPATH SELECTORS

A selector that encodes the DOM's exact shape today is a selector that breaks tomorrow.

No nth-child Chains

div.container > div:nth-child(3) > button breaks the instant a designer reorders or adds a sibling element.

No Deep Structural Paths

A selector spanning five nested levels ties the test to internal structure that has nothing to do with its actual intent.

Prefer Semantic, Scoped Selectors

Scope from a stable data-testid'd ancestor, then use a short, semantically meaningful child selector.

KEY TAKEAWAY Every fragile selector this module warns about shares one root cause: encoding incidental DOM structure instead of deliberate, test-facing intent.

TRY IT YOURSELF — EXERCISE 4: CHOOSE STABLE SELECTORS

Reinforces: Selenium architecture, WebDriver, Angular DOM rendering, and the stable-versus-fragile selector slides you just covered.

STEPS (notes/lab19-selectors.md)

Step	What To Do
1 — Selector Table	For the customer list, row, and save button, give the selector you would use.
2 — Why Not CSS Paths	Say what breaks a selector like div > div:nth-child(3) > span.
3 — Angular Classes	Say why Angular's generated attributes make poor hooks.
4 — Add the Hook	Name the attribute the team adds to templates for testing.

Selector choices

```
list    [data-testid='customer-list']
row     [data-testid='customer-row-CUS-1001']
avoid   div > div:nth-child(3) > span   and   ._ngcontent-abc-12
```

TRY IT NOW — Complete Exercise 4 before continuing: exercises/exercise-04-selector-strategy.md (workspace: examples/module-19-exercises).

AVOIDING FRAGILE CSS/XPATH SELECTORS

Fragile selectors break when UI structure, classes, or layout changes. Use resilient selectors for stable and reliable tests.

WHY FRAGILE SELECTORS ARE A PROBLEM

Tightly coupled to UI structure
Break when HTML hierarchy or classes change.

Depend on presentation
CSS classes, positions, and styles often change during UI updates.

Hard to maintain
Complex selectors are difficult to read and update.

Cause flaky tests
Small UI changes lead to unnecessary test failures.

FRAGILE CSS SELECTORS TO AVOID

Problem	Example (Fragile)
Using long class chains	`.app-container .row .col-12 .card .card-body .btn.btn-primary`
Using nth-child / nth-of-type	`ul.menu > li:nth-child(3) > a`
Using generic element with position	`div.container > div > input[type="text"]`
Coupling to layout or styling classes	`.mt-3 .mb-2 .text-center .form-control`
Using dynamic or auto-generated classes	`.css-1abcde-xyz123` `.chakra-xyz-123`

FRAGILE XPATH PATTERNS TO AVOID

Problem	Example (Fragile)
Using absolute XPath	`/html/body/div[1]/div[2]/form/input`
Using indexes everywhere	`//div[3]//ul/li[2]/a`
Deeply nested paths	`//div[@class='container"]]/div/div/div/ /table/tbody/tr[4]/td[2]/button`
Matching on text that can change	`//button[text()="Submit']`
Using full paths for simple elements	`//div[2]/div[1]/span[1]`

BEST PRACTICES FOR STABLE SELECTORS

- Prefer IDs, data-test, data-testid, data-qa, or aria-* attributes.
- Use meaningful attribute values that rarely change.
- Avoid relying on CSS classes, layout structure, or element position.
- Keep selectors as short and as specific as possible.
- Work with developers to add stable test attributes in the UI.

```
<button
  type="button"
  data-testid="save-user-btn"
  class="btn btn-primary">
   Save
</button>
```

Good: Stable and resilient selector

REAL-WORLD EXAMPLE: BEFORE vs AFTER

BEFORE (Fragile)

- CSS: `.container > div > div:nth-child(2) > input`
- XPath: `//div[4]/div[2]/span/input`
- Problem: Breaks when layout or structure changes.

AFTER (Stable)

- CSS: `[data-testid="username-input"]`
- XPath: `//input[@data-testid='username-input']`
- Benefit: Stable, readable, and easy to maintain.

Key Takeaway: Avoid fragile CSS/XPath selectors that depend on UI structure, classes, or layout.
Use stable test attributes and meaningful selectors to build reliable and maintainable tests.

WAITING FOR ANGULAR UI STATE

An Angular SPA never fires a traditional page-load event for a route change -- waiting must target UI state, not page load.

No Full Page Reload

driver.get() completing means the HTML shell loaded -- nothing about whether Angular has bootstrapped or fetched its data yet.

Wait for the Specific State

Wait for the exact element or condition the next step depends on -- not a fixed amount of time.

Different Waits for Different States

Waiting for a spinner to vanish differs from waiting for a specific row to appear -- each needs its own condition.

KEY TAKEAWAY The single biggest mindset shift for testing an Angular app with Selenium: stop waiting for 'the page to load' and start waiting for 'the specific thing my next step needs.'

WAITING FOR ANGULAR UI STATE

Angular updates the DOM asynchronously. Your tests must wait for the UI to reach the expected stable state before interacting.

WHY WAIT?

Angular performs asynchronous operations:

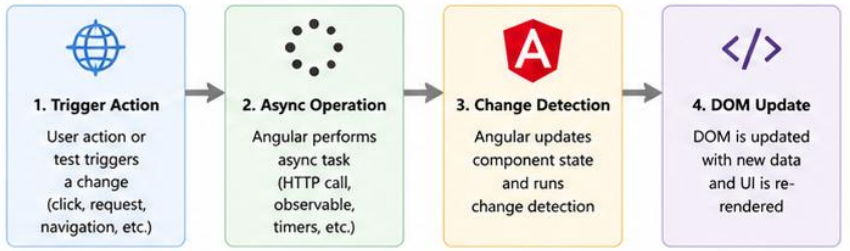
- HTTP requests
- Change detection
- DOM rendering
- Animations
- Route navigation

Interacting too early leads to flaky tests and element not found errors.

GOOD PRACTICES

- Always wait for a specific condition, not a fixed time.
- Wait for elements to be visible and enabled.
- Wait for Angular to be stable (zone is stable).
- Prefer explicit waits over implicit or hard waits.
- Combine waits for better reliability.

ANGULAR UI STATE FLOW



WHAT TO WAIT FOR

Wait For	Description	Example
Element Visible	Element is present in DOM and visible to the user.	`visibilityOf(element)`
Element Clickable	Element is visible and enabled (ready for interaction).	`elementToBeClickable(element)`
Text or Value	Element text/value matches expected value.	`textToBePresentInElement(...)`
Loader / Spinner	Wait until loader disappears or is hidden.	`invisibilityOf(loader)`
Angular Stable	Angular zone is stable (all async tasks complete).	`ngZone.onStable`

TOOLS AND APPROACHES

Implicit Wait

Sets a global max wait time for element search. Simple but less precise.

```
driver.manage().timeouts().implicitlyWait(10, TimeUnit.SECONDS);
```

Explicit Wait

Waits for a specific condition up to a timeout.

```
WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(10));
wait.until(ExpectedConditions.visibilityOf(element));
```

Angular Stability

Wait until Angular has no pending async tasks.

```
wait.until(driver ->
((JavascriptExecutor) driver).executeScript(
"return window.getAllAngularTestabilities().every(t => t.isStable())"));
```

PRO TIP

Combine waits: wait for Angular stable + element visible.
This reduces flakiness in real-world Angular applications.

```
wait.until(driver -> ((JavascriptExecutor) driver)
    .executeScript("return window.getAllAngularTestabilities()
    .every(t => t.isStable())"));
wait.until(ExpectedConditions.visibilityOf(element));
```

AVOID

```
Thread.sleep(...)          // Causes slow and flaky tests
wait for long times        // Increases test execution time
Relying only on implicit waits   // Hides real synchronization issues
```

Key Takeaway: Always wait for the UI to reach a stable state before interacting.
Use explicit waits and Angular stability checks to write reliable, maintainable tests.

EXPLICIT WAITS

WebDriverWait plus ExpectedConditions is the correct, reliable way to wait -- Thread.sleep() is not.

WebDriverWait

new WebDriverWait(driver, Duration.ofSeconds(10)) polls repeatedly, up to a timeout, instead of blocking for a fixed duration.

ExpectedConditions

visibilityOfElementLocated, elementToBeClickable, and invisibilityOfElementLocated cover nearly every Angular wait scenario.

Never Thread.sleep()

A fixed sleep is always either wastefully slow or unreliably fast -- it should never appear in this course's Selenium tests.

KEY TAKEAWAY Explicit waits are the fundamental missing piece: wait for a specific condition, with a bounded timeout, instead of guessing at a fixed delay.

EXPLICIT WAITS

Explicit waits pause test execution until a specific condition is met or a timeout occurs.

WHY EXPLICIT WAITS?

Angular updates the DOM asynchronously.
Elements may not be immediately available.

Without waits, tests may fail randomly
causing flaky and unreliable tests.

Explicit waits wait for the exact condition
you need, making tests stable and faster.

Better control over timeout and polling
improves performance and reliability.

HOW EXPLICIT WAIT WORKS

1. Start Waiting
Test waits until the condition is true or timeout occurs.

2. Poll Condition
Selenium polls the condition at a set interval (default 500ms).

3. Condition Met
If condition becomes true, WebDriver proceeds immediately.

4. Timeout
If condition not met within timeout, TimeoutException is thrown.

EXAMPLE: BASIC EXPLICIT WAIT

```
WebDriver driver = new ChromeDriver();
WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(10));

// Wait until element is visible
WebElement saveBtn = wait.until(
   ExpectedConditions.visibilityOfElementLocated(
      By.id("save-user-btn")
   )
);
saveBtn.click();
```

COMMON EXPECTED CONDITIONS

Condition	Description
`visibilityOf(element)`	Element is present in DOM and visible to the user.
`elementToBeClickable(element)`	Element is visible and enabled so it can be clicked.
`invisibilityOf(element)`	Element is not visible in DOM or has height/width = 0.
`presenceOfElementLocated(locator)`	Element is present in the DOM (regardless of visibility).
`textToBePresentInElement(element, text)`	Element's text contains the expected text.
`stalenessOf(element)`	Element is detached from the DOM (no longer valid).

EXPLICIT WAIT SYNTAX

```
WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(10));
WebElement element = wait.until(
    ExpectedConditions.<condition>(By.<locator>("value"))
);
```

- **Timeout:** Maximum time to wait (e.g., 10 seconds)
- **Polling Interval:** How often condition is checked (default 500ms)

EXAMPLE USE CASES

- Wait for loading spinner to disappear
- Wait for data to load in a table
- Wait for success or error message
- Wait for element to become clickable
- Wait for route/navigation to complete

BEST PRACTICES

- Use explicit waits instead of `Thread.sleep()`.
- Wait for specific conditions, not just time.
- Use the shortest timeout that is reliable.
- Combine explicit waits with stable selectors.
- Do not mix implicit and explicit waits.

Key Takeaway: Explicit waits wait for a specific condition to be true before continuing.
They are essential for stable, reliable, and maintainable Selenium tests with Angular applications.

TESTING ANGULAR FORMS

A Reactive Form's validation state is reflected in the DOM -- Selenium can assert against it, the same as any other rendered state.

Fill Every Field

sendKeys() into each input; clear() first if a field might already hold a stale value from a previous step.

Assert Disabled Submit

A Reactive Form's invalid state often disables the submit button -- assert isEnabled() before and after fixing a field.

Assert Validation Messages

A required-field error message appearing (or disappearing after a fix) is itself a testable, meaningful UI state.

KEY TAKEAWAY Testing a Reactive Form end to end means testing the same validation feedback loop covered in the Frontend-to-API-Integration material -- Selenium just observes it from the outside, through the browser.

TESTING ANGULAR FORMS

Angular forms are dynamic and stateful, Tests should verify input, validation, submission, and UI feedback.

WHY TEST FORMS?

- Forms capture critical user input.
- Validation rules change often.
- Prevent invalid data from reaching the API.
- Ensure a smooth user experience.

TYPES OF ANGULAR FORMS

Template-Driven Forms

Easier for simple forms.

`</> FormsModule`

Reactive Forms

More powerful, scalable and testable.

`</> ReactiveFormsModule`

KEY AREAS TO TEST

- Rendering of form controls
- User input and model binding
- Validation (required, pattern, min/max, custom)
- Error message display
- Form submission (valid & invalid)
- Disabled state / Loading state

REACTIVE FORM EXAMPLE

```
this.employeeForm = this.fb.group({
  name: ['', [Validators.required, Validators.minLength(3)]],
  email: ['', [Validators.required, Validators.email]],
  department; ['', Validators.required]
});

onSubmit() {
  if (this.employeeForm.valid) {
    this.saveEmployee(this.employeeForm.value);
  }
}
```

TESTING WITH TESTBED & ANGULAR TESTING UTILITIES

```
it('should show validation error when name is empty', () => {
   component.employeeForm.controls['name'].setValue('');
   component.employeeForm.controls['name'].markAsTouched();
   fixture.detectChanges();

   const errorEl = fixture.nativeElement.querySelector('.error-name');
   expect(errorEl.textContent).toContain('Name is required');
});

it('should submit form when valid', () => {
   spyOn(component, 'saveEmployee');
   component.employeeForm.setValue({
      name: 'John Doe',
      email: 'john@example.com',
      department: 'IT'
   });
   component.onSubmit();
   expect(component.saveEmployee).toHaveBeenCalled();
});
```

SAMPLE EMPLOYEE FORM (UI)

Name *

Enter name

Name is required (min 3 characters)

Email *

Enter email

Valid email is required

Department *

Select department

Department is required

Save | Cancel

TYPICAL TEST SCENARIOS

1. Load form → verify all controls are visible
2. Leave required fields empty → verify validation messages
3. Enter invalid email → verify email validation error
4. Enter valid data → verify errors disappear
5. Click Save with invalid data → submission should not occur
6. Click Save with valid data → API/service is called

Key Takeaway: Test Angular forms end-to-end: input, validation, and submission.
This prevents invalid data, reduces bugs, and ensures a great user experience.

SIMULATING USER INPUT

sendKeys(), click(), and the Select helper cover the vast majority of realistic user interactions.

Text Input

element.sendKeys("Amina") types character by character, exactly as a real user typing would, triggering Angular's input events.

Dropdowns

The Select helper drives a native <select>. For Angular Material, click and select the option element directly instead.

Checkboxes & Radios

click() toggles a checkbox or radio -- always assert isSelected() rather than assuming the click succeeded.

KEY TAKEAWAY sendKeys() firing real keystroke-level input events matters for Angular specifically, since Angular's reactive forms listen for genuine input events, not just a final value being set.

SIMULATING USER INPUT

Simulate how real users type, select, check, and interact with UI controls in Angular applications.

WHAT IS USER INPUT SIMULATION?

Automated tests replicate real user actions to interact with the UI.

- Typing in text fields
- Selecting dropdown options
- Checking / unchecking checkboxes
- Clicking buttons
- Uploading files
- Triggering events (blur, change, submit)

TYPES OF USER INPUT ACTIONS

Action	Description
`sendKeys()`	Types text into input fields.
`click()`	Clicks buttons, links, checkboxes, etc.
`clear()`	Clears existing text in input fields.
`selectByVisibleText()`	Selects dropdown by visible text.
`selectByValue()`	Selects dropdown by value attribute.
`isSelected()`	Checks if checkbox/radio is selected.
`upload File`	Simulates file upload input.

SAMPLE ANGULAR FORM

Name *
Enter your name

Email *
Enter your email

Department
Select department

Role
- Employee
- Manager
- Admin

Skills
- [x] Java
- [x] Angular
- [ ] PostgreSQL
- [ ] Kafka

Resume
Choose File | No file chosen

Save Employee | Reset

* Required fields

COMMON INPUT VALIDATIONS TO TEST

- Required field validation
- Email format validation
- Min / Max length validation
- Dropdown selection validation
- Checkbox selection rules
- File type / size validation

EXAMPLES: SIMULATING USER INPUT (SELENIUM + ANGULAR)

```
WebDriver driver = new ChromeDriver();
WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(10));

// 1. Type text into input field
WebElement nameInput = wait.until(ExpectedConditions.visibilityOfElementLocated(By.id("name")));
nameInput.clear();
nameInput.sendKeys("John Doe");

// 2. Type email
WebElement emailInput = driver.findElement(By.cssSelector("input[formControlName='email']"));
emailInput.sendKeys("john.doe@example.com");

// 3. Select dropdown by visible text
WebElement deptDropdown = driver.findElement(By.id("department"));
Select deptSelect = new Select(deptDropdown);
deptSelect.selectByVisibleText("Engineering");

// 4. Select multiple checkboxes
driver.findElement(By.id("skill-java")).click();
driver.findElement(By.id("skill-angular")).click();

// 5. Select radio button
driver.findElement(By.id("role-employee")).click();

// 6. Upload file
WebElement resumeInput = driver.findElement(By.id("resume"));
resumeInput.sendKeys("C:\\TestFiles\\resume.pdf");

// 7. Click Save
driver.findElement(By.cssSelector("button[type='submit']")).click();
```

BEST PRACTICES

- Use stable selectors (id, data-test attributes).
- Clear fields before entering new text.
- Wait for elements to be visible and enabled.
- Simulate real user behavior as closely as possible.
- Validate UI feedback after input (errors, success messages).
- Keep tests independent and idempotent.

AVOID

- Using Thread.sleep() for waiting.
- Typing too fast for dynamic fields without waits.
- Hard-coded test data that makes tests fragile.

QUICK REFERENCE

Method	Purpose
`sendKeys("text")`	Types text
`click()`	Clicks element
`clear()`	Clears text
`selectByVisibleText("Text")`	Selects dropdown by text
`selectByValue("value")`	Selects dropdown by value
`isSelected()`	Checks checkbox/radio state

Key Takeaway: Simulating user input accurately validates how real users interact with your Angular application. It ensures reliability, catches UI issues early, and improves overall test quality.

TESTING BUTTONS AND EVENTS

A click is only half the test -- the resulting event's effect on the UI is what actually needs asserting.

Click and Observe

click() simulates the mouse action; the real assertion is what changes afterward -- a new element, a route change, a spinner.

Expect an Async Effect

Most buttons in this CRM trigger an HTTP call -- wait for the resulting state, never assert immediately after click().

Assert Disabled State Too

A button correctly disabling itself while a request is in flight is itself testable UI behavior, not just a styling detail.

KEY TAKEAWAY Never assert immediately after a click() call in this Angular application -- always wait for the specific downstream effect that click was supposed to trigger.

MODULE 19

Testing Buttons and Events

Verify that UI buttons trigger the correct events and application behavior.

What We Test

Buttons trigger the correct click events and the application responds as expected.

What to Verify

- Click handler is triggered
- UI state changes as expected
- Navigation occurs (if applicable)
- API calls are made (when relevant)
- API calls are made (when relevant)
- Success / error messages are displayed

Best Practices

- Use stable selectors (id, data-test)
- Wait for UI state changes
- Assert on visible outcomes, not implementation

User Click Flow

1. Find Button (Element)
2. Click (Element)
3. Event Handler Triggered
4. UI / State Updates

Selenium WebDriver Example (TypeScript)

```
// Find button using a stable selector
const saveButton = driver.findElement(By.id('btn-save'));

// Click the button
await saveButton.click();

// Assert success message is displayed
const successMsg = await driver.findElement(
   By.css('[data-test="save-success"]')
);
await expect(successMsg).toBeDisplayed();

// Assert navigation occurred
await expect(driver.getCurrentUrl()).toContain('/employees');
```

Pro Tip

Avoid hardcoding sleeps. Use Explicit Waits to wait for expected UI changes after button clicks.

Example Scenarios

- **Save Button** Triggers save, displays success message
- **Delete Button** Opens confirm dialog, then performs delete
- **Add Button** Opens form or navigates to create page
- **Back Button** Navigates to previous page/view
- **Refresh Button** Reloads data and updates the UI

TESTING NAVIGATION

The Angular Router changes the URL and swaps components without a full page reload -- assert both the URL and the new component's content.

Assert the URL

driver.getCurrentUrl() should reflect the new route after a navigation action -- e.g. ending in /customers/CUS-1001.

Assert the New Component Rendered

The URL changing alone doesn't guarantee the new route's component actually finished rendering its content.

Test Back/Forward Too

driver.navigate().back() should return to the previous Angular route correctly, not break the SPA's state.

KEY TAKEAWAY A green navigation test that only checks the URL, and never checks that the destination route's content actually rendered, is not a complete navigation test.

MODULE 19

Testing Navigation

Verify that users can navigate through the application and the correct page loads.

What We Test

Navigation links, menu items, and route changes load the correct view and maintain expected state.

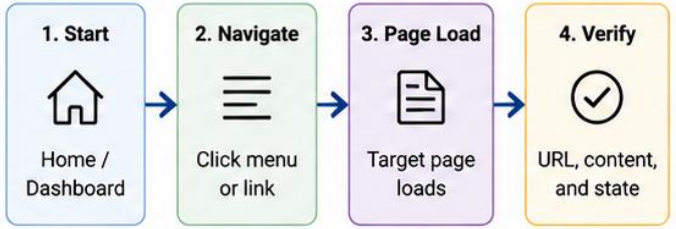
What to Verify

- Correct page/view is displayed
- URL changes as expected
- Active menu/route is highlighted (if applicable)
- Breadcrumbs update (if applicable)
- Data on the page is loaded correctly
- Back/forward navigation works (if applicable)

Best Practices

- Use stable selectors (id, data-test, aria-label)
- Assert on visible outcomes, not implementation
- Wait for navigation to complete before asserting
- Keep tests independent and isolated

Navigation Flow (Example)



What We Verify (Examples)

- Click "Employees" → Employees list page is displayed
- Click "Add Employee" → Employee form page is displayed
- Breadcrumb shows: Home > Employees > Add
- URL contains /employees/add
- Active menu item "Employees" is highlighted

Selenium WebDriver Example (TypeScript)

```typescript
// Click on Employees menu item
await driver.findElement(By.css('[data-test="nav-employees"]').click();

// Wait until Employees page is loaded
await driver.wait(until.urlContains('/employees'), 10000);

// Assert page title
await expect(driver.getTitle()).resolves.toContain('Employees');

// Assert active menu
const active = await driver.findElement(
   By.css('[data-test="nav-employees"].active')
);
await expect(active).toBeDisplayed();

// Navigate to Add Employee
await driver.findElement(By.css('[data-test="btn-add"]')).click();

// Verify Add Employee page
await expect(driver.getCurrentUrl()).toContain('/employees/add');
await expect(driver.findElement(By.css('h1'))))
    .toHaveText('Add Employee');
```

Pro Tip

Use Explicit Waits to ensure navigation has completed before validating the next page. Avoid hardcoded sleeps.

Example Navigation Scenarios

- **Menu Navigation** — Navigate through top/side menus
- **Deep Link Access** — Open a page directly using a URL
- **Back Navigation** — Use browser back to return to previous page
- **Browser Refresh** — Refresh page and verify state persists
- **Cross Page Flow** — Complete a multi-step user journey

TESTING REST-BACKED ANGULAR SCREENS

Nearly every screen in this CRM is a thin Angular shell around data from the Spring Boot API -- test the screen against known, seeded fixture data.

Same Fixtures as the Backend

Assert against the same CUS-1001/Amina fixture data already seeded for backend tests -- one shared source of truth.

Data Arrives Asynchronously

The screen renders its shell immediately but its real content only appears once the HTTP response comes back.

Retest After Any State Change

Creating or deleting a customer should be followed by a fresh assertion against the updated screen, not an assumption.

KEY TAKEAWAY A Selenium test against a REST-backed screen is really testing three things at once -- the Angular UI, the HTTP call, and the Spring Boot API's response -- so seeded, known fixture data is what makes its assertions meaningful.

MODULE 19

Testing REST-Backed Angular Screens

Verify that Angular screens correctly call REST APIs and display data, loading, empty, and error states as expected.

What We Test

Angular screens that consume REST APIs for data display and user actions (CRUD).

What to Verify

- API calls are triggered with correct method, URL, headers, and payload
- Successful responses render data correctly
- Loading indicators appear and disappear
- Empty state is shown when no data is returned
- Error state is shown on API failures
- User actions (create, update, delete) refresh data

Best Practices

- Use stable selectors (id, data-test)
- Intercept and mock APIs when appropriate
- Assert on visible UI outcomes, not implementation
- Keep tests independent and deterministic

Screen Data Flow

1. **User Action** — User opens screen or performs action
2. **API Request** — Angular service calls REST API
3. **API Response** — Backend returns JSON data
4. **UI Update** — Component updates UI with data
5. **Verify** — Assert data, state, and behavior

What We Verify (Examples)

- Employee list screen loads data from GET /api/employees
- Create Employee form submits data via POST /api/employees
- Employee details screen loads via GET /api/employees/{id}
- Update employee triggers PUT /api/employees/{id}
- Delete employee triggers DELETE /api/employees/{id}
- Empty state shown when response is an empty array
- Error message shown when API returns 4xx/5xx

Selenium WebDriver Example (TypeScript)

```typescript
// Intercept API call
cy.intercept('GET', '/api/employees*').as('getEmployees');

// Navigate to Employees screen
cy.visit('/employees');

// Wait for API call and response
cy.wait('@getEmployees');

// Assert data rendered
cy.get('[data-test="employee-row"]').should('have.length.greaterThan', 0);

// Verify employee name is visible
cy.get('[data-test="employee-row"] [data-test="employee-name"]')
  .first().should('be.visible');

// Verify loading indicator disappears
cy.get('[data-test="loading-spinner"]').should('not.exist');

// Example: Verify empty state
cy.intercept('GET', '/api/employees*', { body: [] }).as('emptyEmployees');
cy.visit('/employees');
cy.wait('@emptyEmployees');
cy.get('[data-test="empty-state"]').should('be.visible');

// Example: Verify error state
cy.intercept('GET', '/api/employees*', { statusCode: 500 }).as('errorEmployees');
cy.visit('/employees');
cy.wait('@errorEmployees');
cy.get('[data-test="error-message"]').should('be.visible');
```

Pro Tip

Use `cy.intercept()` to control API responses and simulate success, empty, and error scenarios for reliable UI tests.

Example REST-Backed Screen Scenarios

- **List Screen** — Load and display paginated data
- **Create Screen** — Submit form and show new item
- **Update Screen** — Edit item and reflect changes
- **Delete Action** — Remove item and refresh list
- **Filter/Search** — Apply filter and show matching results

HANDLING LOADING INDICATORS

Wait for the loading spinner to disappear -- not for a fixed delay, and not for the spinner to appear first (it may already be gone by the time Selenium checks).

invisibilityOfElementLocated

The standard ExpectedCondition for 'the spinner is gone, content should be ready now.'

It May Never Even Appear

On a fast run, a spinner can appear and vanish faster than Selenium can observe it -- don't require seeing it appear first.

Wait for Content, Not Just Spinner Absence

The safest condition is often 'the specific data element is visible,' which implies the spinner is gone too.

KEY TAKEAWAY A test that waits for the spinner to appear before waiting for it to disappear is a test that can fail on a fast machine and pass on a slow one -- wait only for the state you actually need.

MODULE 19

Handling Loading Indicators

Verify that loading indicators appear during async operations and disappear when data is loaded or an error occurs.

What We Test

Loading indicators (spinner, progress bar, skeleton, etc.) behave correctly during API calls.

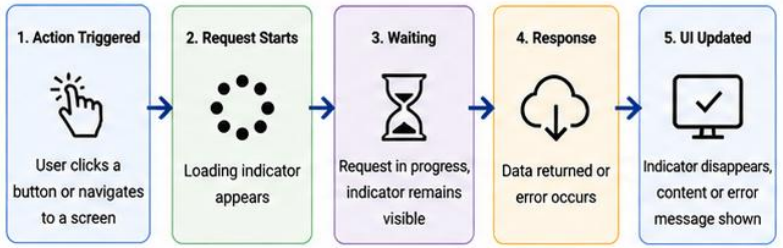
What to Verify

- Indicator is visible when request starts
- Indicator remains visible while request is in progress
- Indicator disappears when request succeeds or fails
- Data or error message is shown after indicator disappears
- UI remains responsive (no frozen state)

Best Practices

- Use stable selectors (id, data-test)
- Use Explicit Waits for visibility changes
- Assert on user-visible outcomes
- Test success, empty, and error scenarios

Loading Indicator Flow



What We Verify (Examples)

- Click "Load Employees" → spinner becomes visible
- Spinner remains visible while API call is in progress
- Spinner disappears after data is loaded
- Employee list is displayed after spinner disappears
- When API fails, spinner disappears and error message is shown
- No spinner remains visible when leaving the screen

Selenium WebDriver Example (TypeScript)

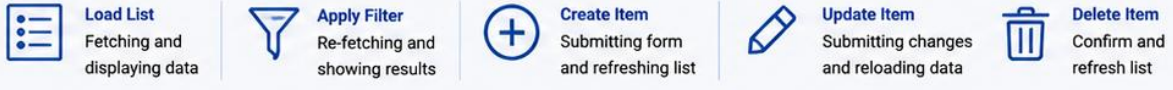
```typescript
// Trigger request (e.g., click Load button)
await driver.findElement(By.css('[data-test="load-employees"]')).click();

// 1. Wait for spinner to appear
const spinner = await driver.findElement(By.css('[data-test="loading-spinner"]'));
await expect(spinner).toBeDisplayed();

// 2. Wait for spinner to disappear
await driver.wait(untli.invisibilityOf(spinner), 15000);

// 3. Verify data is loaded
await expect(driver.findElements(By.css('[data-test="employee-row"]')))
        .toHaveLength.greaterThan(0);

// Error scenario example
await driver.findElement(By.css('[data-test="load-employees"]')).click();
await driver.wait(untli.invisibilityOf(spinner), 15000);
const error = await driver.findElements(By.css('[data-test="error-message"]'));
await expect(error[0]).toBeDisplayed(();
```

Pro Tip

Always wait for the spinner to disappear before asserting on the final UI state. Avoid hardcoded sleeps.

Common Loading Indicator Types

Example Scenarios



TESTING ERROR STATES

A Selenium test can deliberately trigger a backend error -- a bad ID, invalid input -- and assert the Angular UI shows it correctly.

Trigger a Known Failure

Navigate to a customer detail URL with a nonexistent ID to reliably trigger the 404 empty/error state.

Assert the Error Message Renders

Wait for the specific error-state element (matching this course's UI-states discipline) to become visible.

Assert No Stale Content Lingers

The error state should replace any loading indicator or stale prior content -- not render alongside it.

KEY TAKEAWAY Testing error states means intentionally causing a failure and proving the Angular UI degrades gracefully -- this is what separates a demo-quality integration from the production-ready one this course aims for.

TRY IT YOURSELF — EXERCISE 5: PLAN WAITS AND UI STATES

Reinforces: waiting for Angular UI state, explicit waits, forms, navigation, loading indicators, and error-state testing as you just covered.

STEPS (notes/lab19-waits.md)

Step	What To Do
1 — Sleep vs Wait	Say why Thread.sleep(3000) is worse than an explicit wait, in both directions.
2 — Wait Conditions	Give the condition to wait on for the list, and for the spinner disappearing.
3 — Error State Test	Describe a test that proves the error banner appears, not just that nothing crashed.
4 — Form Journey	List the steps for creating a customer through the UI.

Explicit waits

```
wait.until(visibilityOfElementLocated(byTestId('customer-list')))
wait.until(invisibilityOfElementLocated(byTestId('loading')))
never: Thread.sleep(3000)  -- too slow when fast, too short when slow
```

TRY IT NOW — Complete Exercise 5 before continuing: exercises/exercise-05-waits-and-states.md (workspace: examples/module-19-exercises).

MODULE 19

Testing Error States

Verify that the application handles errors gracefully and displays clear, user-friendly messages.

What We Test

UI behavior when API calls fail or return errors, and when client-side validation fails.

What to Verify

- Error message is displayed
- Correct error message content
- UI elements remain usable
- No broken or unhandled states
- Retry or corrective actions are available (if applicable)

Best Practices

- Trigger real error scenarios (mock or test env)
- Assert on visible outcomes, not internal errors
- Use stable selectors (id, data-test)
- Keep tests independent and deterministic

Common Error Scenarios

Network / Server Error
e.g., 500, 503
Show friendly error message and retry option.

Unauthorized
e.g., 401, 403
Show access error and redirect to login if needed.

Not Found
e.g., 404
Show "not found" message or empty state.

Validation Error
e.g., 400
Show field-level validation messages clearly.

What We Verify (Examples)

- Server returns 500 → Show "Something went wrong. Please try again."
- Unauthorized 401 → Show "Please log in to continue."
- 404 Not Found → Show "Resource not found." or empty state
- Form validation fails → Show field-level validation messages
- Retry button works and hides error on success
- UI remains responsive and no broken layout appears

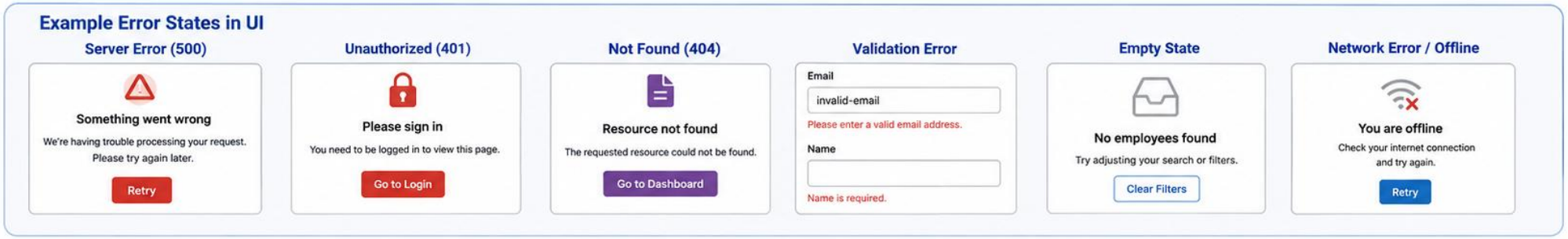
Selenium WebDriver Example (TypeScript)

```typescript
// Trigger error scenario (e.g., intercept API to return 500)
cy.intercept('GET', '/api/employees', { statusCode: 500}).as('getEmployeesError');

// Navigate to Employees screen
cy.visit('/employees');

// Wait for error message to appear
cy.wait('@getEmployeesError');
cy.get('[data-test="error-message"]').should('be.visible')
  .and('contain.text', 'Something went wrong');

// Verify retry button is visible
cy.get('[data-test="retry-button"]').should('be.visible');

// Click retry and wait for success
cy.intercept('GET', '/api/employees', { fixture: 'employees.json' })
   .as('getEmployeesSuccess');
cy.get('[data-test="retry-button"]').click();
cy.wait('@getEmployeesSuccess');
cy.get('[data-test="employee-row"]').should('have.length.greaterThan', 0);
```

Pro Tip

Control API responses with `cy.intercept()` to simulate error scenarios reliably. Always assert on the final UI state and user-visible messages.

Example Error States in UI



AUTHENTICATION FLOW TESTING

Login, protected-route redirect, and logout are the three Selenium scenarios that prove this course's Angular auth flow genuinely works end to end.

Successful Login

Fill valid credentials, submit, and wait for the redirect to the authenticated route -- proving the JWT round trip works.

Protected Route Redirect

Navigating directly to a protected URL while logged out should redirect to /login, proving the route guard is genuinely wired.

Logout Clears Session

After logout, navigating back to a protected route should redirect to login again -- proving no stale token remains usable.

KEY TAKEAWAY Login, protected-route redirect, and logout together prove the whole chain -- login form, JWT, auth interceptor, and route guard -- genuinely works, not just each piece in isolation.

MODULE 19

Authentication Flow Testing

Verify that users can authenticate successfully and receive proper access based on roles and token validation.

What We Test

The login, token issuance, protected resource access, and logout flows.

What to Verify

- Valid credentials → login success & token issued
- Invalid credentials → error message
- Access to protected endpoints with valid token
- Access denied (401/403) without or with invalid token
- Role-based access (e.g., ADMIN vs USER)
- Logout invalidates token (if applicable)
- Token expiration handling

Best Practices

- Use test users with specific roles
- Store token securely during test execution
- Do not hardcode passwords or secrets
- Clean up sessions and test data
- Test both positive and negative scenarios

Authentication Flow (Example)

1. **Login Request** — User submits username & password
2. **Validate Credentials** — Server validates credentials
3. **Issue Token** — Server returns JWT access token
4. **Access Protected API** — Client calls API with token in Authorization header
5. **Validate Token** — Server validates token (signature, expiry, roles)
6. **Response** — Server returns data or access denied
7. **Logout** — Client logs out and token is invalidated (if supported)

What We Verify (Examples)

- Login with valid credentials → receives 200 and JWT token
- Login with invalid credentials → receives 401 and error message
- Access protected endpoint with valid token → receives 200
- Access protected endpoint without token → receives 401
- Access endpoint with insufficient role → receives 403
- Expired token → receives 401 and "token expired" message
- Logout → token becomes invalid (subsequent calls fail)

Role-Based Access Example

Role	Allowed Endpoints
ADMIN	/api/employees/** /api/admin/**
USER	/api/employees/view /api/profile
GUEST	/api/public/**

Pro Tip

Use environment variables for test credentials and avoid committing secrets.

Reuse tokens within a test run to reduce login calls, but test expiration handling.

Selenium WebDriver Example (TypeScript)

```typescript
// 1. Perform login and capture token
const loginResp = await request.post('/api/auth/login')
  .send({ username: 'testuser', password: 'Password123!' });
expect(loginResp.status()).toBe(200);
const token = loginResp.body().token;

// 2. Access protected endpoint with token
const apiResp = await request.get('/api/employees')
  .set('Authorization', `Bearer ${token}`);
expect(apiResp.status()).toBe(200);

// 3. Access without token (should fail)
const noTokenResp = await request.get('/api/employees');
expect(noTokenResp.status()).toBe(401);

// 4. Access with expired/invalid token
const invalidResp = await request.get('/api/employees')
  .set('Authorization', 'Bearer invalid-token');
expect(invalidResp.status()).toBe(401);

// 5. Role-based access validation (expect 403)
const userToken = await loginAs('normaluser');
const adminResp = await request.get('/api/admin/dashboard')
  .set('Authorization', `Bearer ${userToken}`);
expect(adminResp.status()).toBe(403);
```

Example Test Scenarios

- **Successful Login** — Valid credentials return token and user info
- **Invalid Login** — Wrong credentials return 401 and error message
- **Protected API Access** — Valid token allows access to protected resources
- **Unauthorized Access** — No/invalid token returns 401 Unauthorized
- **Insufficient Role** — User without role receives 403 Forbidden
- **Logout Flow** — After logout, token is invalid and access is denied
- **Token Expiry** — Expired token returns 401 and proper message

HEADLESS BROWSER EXECUTION

A CI runner has no display -- headless Chrome renders and runs the Angular app without ever opening a visible window.

--headless=new

ChromeOptions.addArguments("--headless=new") runs Chrome fully, just without rendering pixels to an actual screen.

WebDriverManager

WebDriverManager.chromedriver().setup() automatically downloads the chromedriver binary matching the installed Chrome version.

Same Behavior, No Display

A well-written test behaves identically headless or headed -- if not, that's usually a hidden timing bug worth fixing.

KEY TAKEAWAY Headless execution isn't a testing shortcut -- it's the only mode a CI runner can actually use, since there's no physical or virtual display attached to render a visible browser window.

MODULE 19

Headless Browser Execution

Run UI tests in a headless browser (without UI) for faster, stable, and CI-friendly execution.

What We Test

Verify that UI functionality works correctly when the browser runs in headless mode.

What to Verify

- Test cases pass in headless mode
- Page loads and elements are interactable
- No visual rendering is required
- Screenshots and logs are captured on failure
- Consistent results with headed vs headless

Best Practices

- Use headless mode in CI/CD pipelines
- Keep tests independent and deterministic
- Capture artifacts (screenshots, logs, HTML) on failure
- Avoid browser-specific UI interactions (alerts, downloads) in headless
- Use explicit waits instead of Thread.sleep()

Why Use Headless Mode?

Faster	CI Friendly	Resource Efficient	Stable	Scalable
No UI rendering overhead	Runs on servers without display	Lower CPU and memory usage	Reduces flakiness from UI delays	Run more tests in parallel

Headless Options by Browser

Browser	How to Enable Headless	Notes
Chrome	`--headless=new` (Chrome 109+)	Use "--headless=new" for better compatibility
Edge	`--headless=new`	Same as Chrome
Firefox	`-headless`	Supported in recent geckodriver versions
Safari	`--headless` (Safari Technology Preview)	Limited support

Selenium WebDriver Example (Java)

```java
import org.openqa.selenium.chrome.ChromeOptions;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;

public class HeadlessExample {
  public static void main(String[] args) {
    ChromeOptions options = new ChromeOptions();
    options.addArguments("--headless=new");
    options.addArguments("--window-size=1920,1080");
    options.addArguments("--disable-gpu");
    options.addArguments("--no-sandbox");

    WebDriver driver = new ChromeDriver(options);

    driver.get("https://employee-portal.example.com");
    System.out.println("Page Title: " + driver.getTitle());

    // ... test steps ...

    driver.quit();
} }
```

Pro Tip

Set window-size in headless mode to ensure consistent layout and element visibility across environments.

Headless Test Workflow

Write UI Test (Selenium + TestNG/JUnit) → Enable Headless Mode (Browser Options) → Execute on CI/Environment (No UI Required) → Verify Results (Pass/Fail) → Capture Artifacts (Screenshots, Logs, HTML)

Artifacts on Failure

Screenshot | Page Source | Console Logs | Network Logs | Test Reports

RUNNING SELENIUM IN CI

The Selenium suite runs automatically on every pull request -- GitHub Actions, not a developer's laptop, is the source of truth.

GitHub Actions, Not a Laptop

This course's CI runs on GitHub Actions -- every PR gets the same clean, reproducible environment, not whatever's local.

Full Stack Must Be Running

The Spring Boot API and the built Angular app both need to be up before the Selenium suite can exercise either one.

Gate the Merge on Green

A failing Selenium run blocks the merge, the same as a failing unit or integration test suite would.

KEY TAKEAWAY Running Selenium in CI is what turns a UI test suite from 'something a developer remembers to run before a demo' into a real, enforced quality gate.

MODULE 19

Running Selenium in CI

Integrate Selenium UI tests into CI pipelines to automate browser testing and catch UI issues early.

What We Test

Execute Selenium UI tests in CI to validate critical user journeys across supported browsers on every commit or PR.

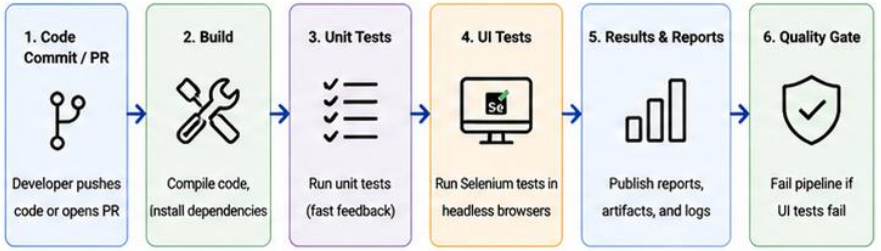
What to Verify

- Tests run automatically on each pipeline trigger
- All critical UI flows are executed
- Tests pass in headless browsers
- Screenshots captured on failure
- Test reports published and accessible
- Pipeline fails when UI tests fail

Best Practices

- Run tests in headless mode
- Keep UI tests stable and independent
- Use explicit waits and resilient selectors
- Parallelize across browsers when possible
- Archive screenshots, videos, and logs
- Mark flaky tests and track them separately

CI Pipeline Flow (Example)



What We Verify (Examples)

- Login flow works with valid credentials
- Create, update, delete employee flows work
- Validation errors are displayed correctly
- Protected routes redirect unauthenticated users
- UI loads correctly across Chrome and Firefox
- No unexpected JavaScript errors in console
- All tests pass in headless mode

Supported Browsers (Headless)

Browser	CI Option
Chrome	--headless=new (Recommended)
Firefox	--headless
Edge	--headless=new
(Optional)	Grid / Docker (Parallel Browsers)

GitHub Actions Example (YAML)

```yaml
name: Selenium UI Tests
on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]
jobs:
  ui-tests:
    runs-on: ubuntu-latest
    strategy:
      matrix:
        browser: [ chrome, firefox ]
    steps:
      - uses: actions/checkout@v4
      - name: Set up JDK
        uses: actions/setup-java@v4
        with:
          distribution: 'temurin'
          java-version: '17'
      - name: Install dependencies
        run: mvn -B clean install -DskipTests
      - name: Run Selenium UI Tests
        run: mvn -B test -Dtest=UITests -Dbrowser=${{ matrix.browser }}
      - name: Upload Surefire Report
        uses: actions/upload-artifact@v4
        with:
          name: surefire-report-${{ matrix.browser }}
          path: target/surefire-reports
      - name: Upload Screenshots on Failure
        if: failure()
        uses: actions/upload-artifact@v4
        with:
          name: screenshots-${{ matrix.browser }}
          path: target/screenshots
```

Artifacts & Reports

- **HTML Report** — View test results in HTML format
- **Screenshots** — Captured on failure
- **Video (Optional)** — Record test execution
- **Logs** — Console and test execution logs
- **Trend Analysis** — Track pass/fail over time
- **Quality Gate** — Block merge/deploy on test failures

Pro Tip

Keep UI tests fast and reliable. Run smoke tests on every PR and full regression on nightly builds.

GITHUB ACTIONS TEST WORKFLOW

A concrete step-by-step shape for the workflow YAML that runs this module's Selenium suite in CI.

WORKFLOW STEP	PURPOSE
actions/checkout@v4	Check out the repository so the workflow has the Angular and Spring Boot source.
actions/setup-java@v4 + actions/setup-node@v4	Provision the JDK for Spring Boot/Maven and Node.js for the Angular build.
ng build --configuration production	Build the Angular app; serve the compiled output for the test run.
mvn spring-boot:run (background) / start the API	Start the Spring Boot API so Selenium has a real backend to call.
mvn verify -Dtest=*SeleniumIT (Failsafe)	Run the headless Selenium suite against the running full stack.

KEY TAKEAWAY Structure the workflow so every dependency the Selenium suite needs -- the built Angular app and a running Spring Boot API -- is genuinely up before the test step runs, and fails the job clearly if either isn't.

GitHub Actions Test Workflow

CI/CD with GitHub Actions

Automated Testing Pipeline for Reliable Code Quality

Goal: Automatically build, test, and validate code on every push or pull request to ensure high-quality, reliable, and production-ready software.

Test Workflow Pipeline

1. **Checkout Code**
   - Checkout repository
   - Fetch latest code from branch/PR
2. **Set Up JDK**
   - Set up Java version
   - Configure Maven environment
3. **Cache Dependencies**
   - Cache Maven dependencies
   - Faster builds on subsequent runs
4. **Run Tests**
   - Execute Maven test phase
   - Run JUnit tests
5. **Publish Test Results**
   - Publish JUnit test results
   - Generate test reports
6. **Upload Artifacts**
   - Upload reports and logs
   - Store build artifacts

Workflow Triggers

- **Push** — Runs on code push to main or feature branches
- **Pull Request** — Runs on PRs before merge
- **Manual Dispatch** — Run workflow manually when needed

Benefits

- **Fast feedback** — Detect issues early
- **Code quality** — Enforce test standards
- **Reliable builds** — Ensure stability before merge
- **Visibility** — Clear reports and artifact tracking

A strong test workflow is the foundation of a trusted CI/CD pipeline.

Example Workflow File: .github/workflows/test.yml

```yaml
name: Java CI with Maven
on: [ push, pull_request ]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
    - uses: actions/checkout@v4
    - uses: actions/setup-java@v4
      with:
        distribution: 'temurin'
        java-version: '17'
    - name: Build and Test
      run: mvn -B clean test
    - name: Publish Test Results
      uses: actions/upload-artifact@v4
      with:
        name: test-reports
        path: target/surefire-reports
```

CAPTURING SCREENSHOTS ON FAILURE

A failure message alone rarely explains what the UI actually looked like -- a screenshot at the moment of failure closes that gap.

TakesScreenshot Interface

((TakesScreenshot) driver).getScreenshotAs(OutputType.FILE) captures the browser's current rendered state as a PNG.

Hook It to Test Failure

A JUnit 5 TestWatcher (or an @AfterEach checking the test result) captures a screenshot automatically, only when a test fails.

Name It Meaningfully

Include the test method name and a timestamp in the filename so a CI failure log can be matched to its screenshot instantly.

KEY TAKEAWAY Debugging a CI-only Selenium failure without a screenshot means guessing at what the browser actually rendered -- capturing one automatically removes the guesswork entirely.

CAPTURING SCREENSHOTS ON FAILURE

Automatically capture UI screenshots when tests fail to speed up debugging and improve issue triage.

HOW IT WORKS

1 Run Tests
Selenium tests execute in the browser.

2 Test Fails
A test assertion fails due to an error.

3 Capture Screenshot
The framework captures a screenshot at the time of failure.

4 Save Artifact
The screenshot is saved with a timestamp and test name.

5 Attach to Report
The screenshot is attached to the test report and CI logs.

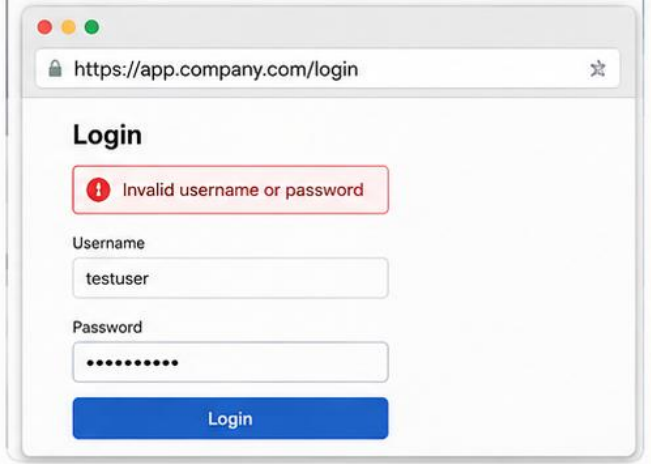
IMPLEMENTATION (Java + Selenium)

```
@AfterEach
public void captureScreenshotOnFailure(ExtensionContext context) {
    if (context.getExecutionException().isPresent()) {
        WebDriver driver = (WebDriver) context.getTestInstance()
            .orElseThrow().get();
        String fileName = context.getDisplayName().replaceAll("[^a-zA-20-9]","_");
        TakesScreenshot ts = (TakesScreenshot) driver;
        File src = ts.getScreenshotAs(OutputType.FILE);
        Files.copy(src.toPath(), Paths.get("screenshots/" + fileName + "_" +
            System.currentTimeMillis() + ".png"), StandardCopyOption.REPLACE_EXISTING);
    }
}
```

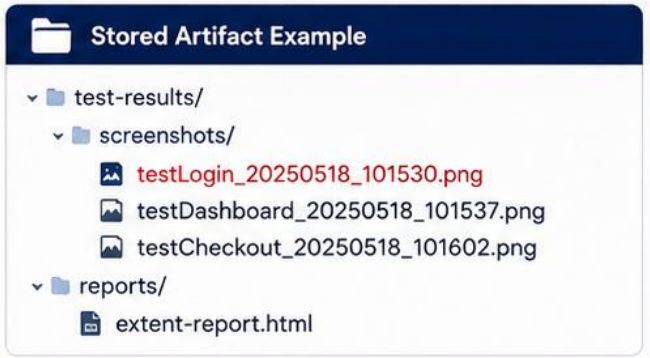
Key Points

- Captured only on failure
- Unique filename with timestamp
- Stored in a dedicated screenshots/ folder
- Automatically attached to test reports

Example Failure Screenshot



Stored Artifact Example



BENEFITS

Faster Debugging
See the exact UI state at the time of failure.

Save Time
Reduces time spent reproducing issues.

Better Reporting
Visual evidence improves test report quality.

Improved Collaboration
Helps teams quickly understand and resolve issues.

TEST REPORTS AND ARTIFACTS

A failing CI run is only useful if a developer can actually see what happened -- reports and artifacts are how.

ARTIFACT	WHERE IT COMES FROM
Surefire/Failsafe XML + HTML reports	Generated automatically by Maven for every unit and integration test run.
Failure screenshots (PNG)	Captured by the TestWatcher pattern from the previous topic.
actions/upload-artifact@v4	The GitHub Actions step that uploads reports and screenshots so they're downloadable from the run.
Console/application logs	Captured alongside test output -- often the first place a root cause actually shows up.
Retention policy	Artifacts typically expire after a set number of days -- long enough to debug, not indefinite storage.

KEY TAKEAWAY A CI failure without an attached artifact is a dead end -- the point of capturing reports, screenshots, and logs is that a developer can diagnose a failure without ever needing to reproduce it locally first.

TEST REPORTS AND ARTIFACTS

Generate, publish, and store test reports and build artifacts to provide visibility, traceability, and faster feedback in the CI/CD pipeline.

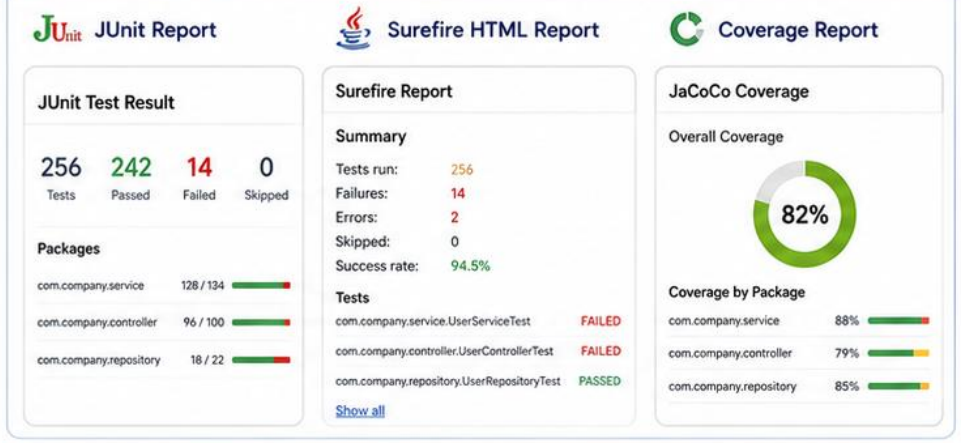
HOW IT WORKS

1. **Run Tests** — Execute unit, integration, and UI tests.
2. **Generate Reports** — Test frameworks generate reports (e.g., JUnit, HTML).
3. **Collect Artifacts** — Reports, logs, screenshots, and build outputs are collected.
4. **Publish Artifacts** — Artifacts are uploaded and stored by the CI platform.
5. **View & Analyze** — Reports are published and available for the team to review.

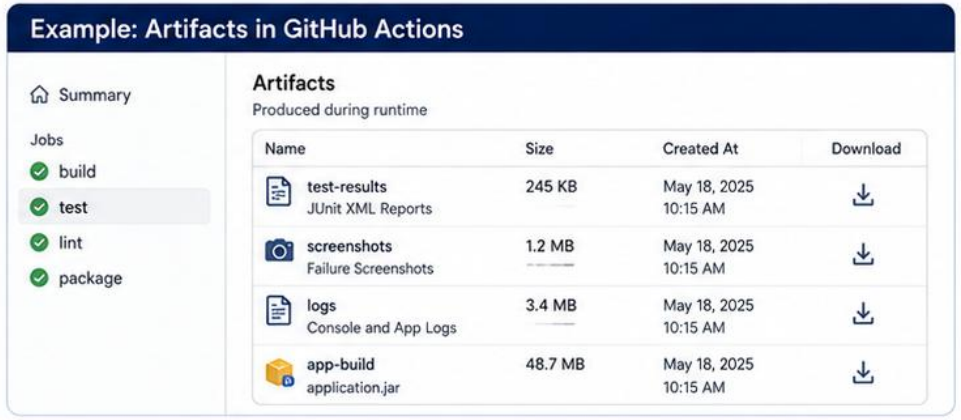
Common Artifacts

Test Reports	Screenshots	Logs	Build Outputs	Other Files
JUnit XML, HTML reports, coverage reports	Captured screenshots on test failures	Application logs, test logs, console outputs	JAR/WAR files, compiled binaries, distribution files	Config files, data files, custom diagnostic outputs

Example Test Reports



Example: Artifacts in GitHub Actions



BENEFITS

- **Better Visibility** — Understand test outcomes and coverage quickly.
- **Faster Debugging** — Artifacts like logs and screenshots speed up root cause analysis.
- **Traceability** — Reports and artifacts provide auditability and history.
- **Team Collaboration** — Share artifacts easily across the development team.

FLAKY TEST CAUSES

The same test passing sometimes and failing other times, with no code change, has a small, well-known list of usual causes.

CAUSE	WHY IT PRODUCES FLAKINESS
Missing or fixed-duration waits	A Thread.sleep() or missing wait races against Angular's real, variable rendering time.
Shared, un-isolated test data	Two tests mutating the same fixture row interfere with each other depending on execution order.
Test execution order dependency	A test that assumes an earlier test already ran (and left state behind) breaks when run alone or reordered.
Environment differences	Headless vs. headed, or CI vs. local, timing and resource differences expose a latent race condition.
Animations and transitions	A CSS transition still in progress can make an element visible-but-not-yet-clickable at the exact instant a click fires.

KEY TAKEAWAY Nearly every flaky Selenium test traces back to one of these five causes -- treat a flaky test as a bug in the test (or a real timing bug it's exposing), never as something to just retry until it passes.

FLAKY TEST CAUSES

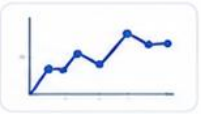
Flaky tests pass or fail inconsistently without code changes, reducing trust in the test suite and slowing down delivery.

COMMON CAUSES

1. **Timing Issues**
   Tests fail due to insufficient or inconsistent waits for UI or API responses.
2. **Async Operations**
   Background calls, animations, or promises not completed before assertions.
3. **Environment Variability**
   Network latency, CPU load, or shared resources cause unpredictable behavior.
4. **Test Data Issues**
   Tests depend on uncontrolled or changing data leading to inconsistent results.
5. **Poor Test Isolation**
   Tests interfere with each other through shared state or side effects.
6. **Fragile Locators**
   Brittle selectors break when UI changes slightly, causing random failures.
7. **UI Changes**
   Frequent UI updates break tests that are tightly coupled to implementation.
8. **External Dependencies**
   Third-party services, APIs, or databases are unreliable or rate limited.
9. **State Leakage**
   Sessions, tokens, cookies, or caches leak between tests.
10. **Resource Constraints**
    Parallel execution exhausts memory, threads, or file handles.

EXAMPLES OF FLAKY TEST BEHAVIOR

Sometimes passes, sometimes fails (the same test run).



Fails more often in CI than locally.

Passes on retry without any code changes.

No clear pattern or consistent root cause.

IMPACT

- Decreases confidence in test results
- Hides real failures
- Slows down development and releases
- Increases maintenance effort

GOOD PRACTICES

- **Use Proper Waits**
  Use explicit waits and synchronize with UI or API states.
- **Isolate Tests**
  Ensure independent test data and clean state before/after each test.
- **Use Stable Locators**
  Prefer IDs, data-attributes, and semantic selectors.
- **Control External Dependencies**
  Mock or stub external services and APIs.
- **Monitor & Retry Wisely**
  Track flaky tests and use limited retries only for known transient issues.

RELIABLE UI TEST BEST PRACTICES

A short, disciplined checklist -- applied consistently -- is what keeps a Selenium suite trustworthy as it grows.

Stable Selectors, Explicit Waits, Always

data-testid selectors and WebDriverWait-based conditions are non-negotiable defaults, not occasional best practice.

Independent, Self-Seeding Tests

Every test seeds and cleans up its own data and never assumes another test ran first.

Keep the Suite Small and Critical

Per the test pyramid, reserve Selenium for the CRM's genuinely critical journeys -- not every possible interaction.

KEY TAKEAWAY Reliable UI test best practices, per this course's testing strategy, are not a separate topic from everything else in this module -- they're the accumulated discipline of every topic already covered, applied consistently.

RELIABLE UI TEST BEST PRACTICES

Follow proven principles to build stable, maintainable, and trustworthy UI tests that deliver consistent results in any environment.

BEST PRACTICES

1. **Use Stable Selectors**
   Prefer IDs, data-* attributes, and ARIA attributes over fragile CSS or XPath.
2. **Wait Intelligently**
   Use explicit waits for conditions. Avoid hard sleeps and implicit waits.
3. **Test User Behavior**
   Interact with the UI like a real user. Avoid bypassing UI logic or APIs.
4. **Keep Tests Focused**
   One test = one goal. Short, simple, and independent tests are easier to debug.
5. **Ensure Test Isolation**
   Reset state before/after tests. Avoid dependencies between tests and shared data.
6. **Handle Dynamic UI**
   Wait for spinners, animations, and network calls to complete.
7. **Assert Meaningfully**
   Use strong assertions on content, state, and behavior—not just element presence.
8. **Capture on Failure**
   Capture screenshots, logs, and page source to speed up root cause analysis.
9. **Keep Locators Resilient**
   Avoid locators tied to layout, index, or text that can change frequently.
10. **Run in Real Conditions**
    Execute in headless and headed modes across browsers and devices in CI.

EXAMPLE: GOOD vs BAD SELECTORS

GOOD	BAD
ID: `#submitBtn`	CSS with classes: `.btn.btn-primary=nth-child(3)`
Data Attribute: `data-testid="login-btn"`	XPath with index: `//div[2]/form/input[1]`
ARIA Attribute: `aria-label="Search"`	Text that can change: `//button[text()='Search Now']`

STABILITY CHECKLIST

- Selectors are stable and intention-revealing
- Tests are independent and repeatable
- Proper waits for dynamic elements and data
- No hard sleeps in tests
- Assertions validate correct behavior
- Tests run consistently in CI and locally
- Screenshots and logs captured on failure
- Tests are maintainable and easy to debug

BENEFITS

- **Consistent Results**
  Reduce flaky failures and increase trust in test outcomes.
- **Faster Feedback**
  Stable tests provide quick, actionable feedback in every build.
- **Lower Maintenance**
  Resilient tests require less rework when UI changes happen.
- **Better CI/CD**
  Reliable tests unblock delivery and improve deployment confidence.
- **Happy Teams**
  Less time debugging tests, more time building great software.

ENTERPRISE TESTING STRATEGY

Bring every layer of this module together into one enforced definition of done for a shippable change.

LAYER	WHAT MUST BE GREEN BEFORE MERGE
Unit tests (Module 18)	Fast, mocked, run on every save -- verify business logic in isolation.
Spring Boot integration tests	MockMvc, Testcontainers-backed repository tests, and TestRestTemplate HTTP tests.
Selenium UI tests	A small, critical set of Angular journeys -- login, create, view -- run headless in CI.
GitHub Actions gate	All three layers run automatically on every pull request; a red run blocks the merge.
Artifacts on failure	Reports, screenshots, and logs are always available to diagnose a failing run without reproducing it locally.

KEY TAKEAWAY None of these five rows is optional, and none substitutes for the others -- an enterprise testing strategy is exactly this stack, enforced consistently, on every single change.

TRY IT YOURSELF — EXERCISE 6: PLAN THE CI STAGE AND SELF-CHECK

Pre-lab gate: plan the headless CI stage and its evidence, then confirm the five earlier notes files are genuinely complete.

STEPS (notes/lab19-ci-readiness.md)

Step	What To Do
1 — Headless Run	Name the browser flags that let Selenium run on a CI runner with no display.
2 — Failure Evidence	Say what is captured when a UI test fails, and where it is uploaded.
3 — Gate Rule	State whether a failing UI test blocks the merge, and why.
4 — Pass Mark	Mark Pass only if all five earlier notes files are complete, not stubbed.

CI stage sketch

```
chrome --headless=new --no-sandbox --window-size=1920,1080
on failure: screenshot + page source -> upload-artifact
UI suite red -> merge blocked; flaky is fixed, never retried away
```

TRY IT NOW — Complete Exercise 6 before Lab 19: exercises/exercise-06-ci-and-readiness.md (workspace: examples/module-19-exercises).

ENTERPRISE TESTING STRATEGY

A balanced, risk-based approach that ensures quality, accelerates delivery, and reduces cost across the software lifecycle.

STRATEGY PRINCIPLES

Quality by Design
Build quality in from the start, not just test it at the end.

Risk-Based Testing
Focus testing effort on critical business risks and high-impact areas.

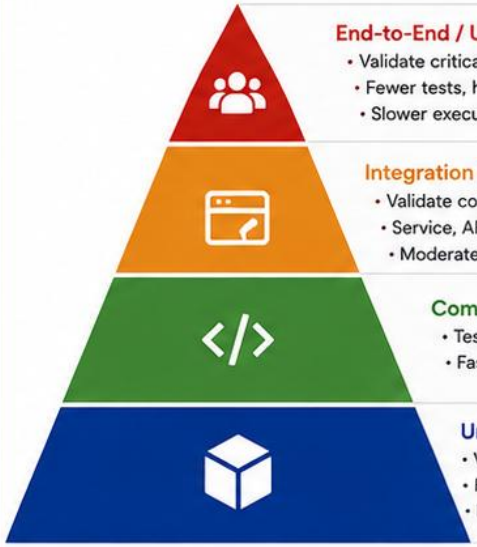
Right Testing at the Right Level
Use the right mix of test types to maximize confidence efficiently.

Automation First
Automate what is repeatable, stable, and provides fast feedback.

Continuous Feedback
Deliver rapid feedback through CI/CD pipelines and quality gates.

End-to-End Quality
Collaborate across teams to ensure quality across the entire stack.

ENTERPRISE TEST PYRAMID



End-to-End / UI Tests — 10%
- Validate critical user journeys
- Fewer tests, higher cost
- Slower execution

Integration Tests — 20%
- Validate component interactions
- Service, API, database, messaging
- Moderate cost and speed

Component / Service Tests — 30%
- Test services, components, APIs
- Fast feedback, good coverage

Unit Tests — 40%
- Validate business logic in isolation
- Fastest execution, cheapest
- High volume, high ROI

TESTING SCOPE ACROSS THE LAYERS

UI / End-to-End
Cross-browser, responsive, accessibility, visual, critical business workflows

Integration
API integration, database integration, message queues, third-party services

Component / Service
Service layer tests, controller tests, validation, mappings

Unit
Functions, methods, calculations, utilities, helpers

QUALITY GATES IN THE PIPELINE

Build → Compile & unit test execution

Static Analysis → Code quality, security, SAST

Automated Tests → Integration & component tests

Artifact Validation → Build artifacts, scan results

Deploy → Promote to next environment

TEST TYPES AND WHEN TO USE

Unit Testing
- Fast feedback
- Developer responsibility
- High volume

Component Testing
- Test in isolation with mocks/stubs
- Service/controller tests

Integration Testing
- Validate interactions between components
- DB, APIs, messaging

End-to-End Testing
- Validate full user flows
- Real browsers
- Business critical paths

Non-Functional Testing
- Performance, security, reliability, usability
- Ensure quality attributes

KEY OUTCOMES

Higher Quality
Defects are caught early and prevented from reaching production.

Faster Delivery
Automation and feedback loops accelerate every release.

Lower Cost
Shift-left testing and automation reduce rework and manual effort.

Lower Risk
Risk-based testing ensures confidence in critical business functionality.

Team Alignment
Shared strategy, standards, and ownership across teams.

STRATEGY SUCCESS FACTORS: ✓ Clear Test Strategy & Standards | ✓ Strong Automation Framework | ✓ Reliable Test Data Management | ✓ Skilled & Empowered Teams | ✓ Continuous Improvement

LAB OVERVIEW — INTEGRATION AND UI TESTING

Regression-test the Northstar CRM application end-to-end, from the Spring Boot API through the Angular UI.

Lab Objectives

Write integration tests for controllers, services, and repositories, then automate Angular UI workflows with Selenium.

Lab Architecture

Spring Boot talks to an isolated PostgreSQL test database while Selenium drives the Angular UI end to end.

Technologies Used

JUnit 5 and Testcontainers back the backend tests; Selenium WebDriver 4.x drives Angular; Allure Report publishes results.

KEY TAKEAWAY This lab's definition of done is a green Spring Boot integration suite and a headless Selenium suite against the Northstar CRM, both publishing Allure/HTML reports from an isolated PostgreSQL test database.

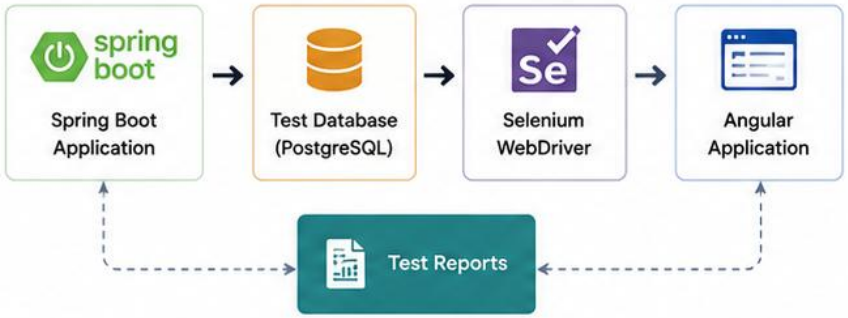
LAB OVERVIEW – INTEGRATION AND UI TESTING

In this lab, you will implement integration tests for a Spring Boot API and automate end-to-end UI tests for an Angular application using Selenium.

LAB OBJECTIVES

- Write integration tests for controllers, services, and repositories.
- Use a test database with isolated test data.
- Automate Angular UI workflows using Selenium WebDriver.
- Validate UI behavior for success, error, and edge cases.
- Run tests in headless mode and generate reports.

LAB ARCHITECTURE



TECHNOLOGIES USED

Spring Boot	Backend application and REST APIs
PostgreSQL	Relational database for test environment
JUnit 5	Testing framework for backend tests
Selenium WebDriver	UI test automation for Angular
Angular	Frontend application under test
Allure Report	Test reporting and visualization

LAB SCOPE

- **Backend:** Controller, Service, Repository integration tests
- **Frontend:** Angular UI automation using Selenium
- Test data setup, isolation, and cleanup
- Headless execution and test reporting
- Capture screenshots on failure

LAB TASKS

1. Set up test database and seed data
2. Write controller integration tests
3. Write service layer integration tests
4. Write repository integration tests
5. Configure Selenium WebDriver
6. Automate Angular UI test scenarios
7. Handle wait conditions and errors
8. Generate and review test reports

KEY SCENARIOS

- Create a new employee (UI + API)
- View employee list with pagination
- Update employee details
- Delete an employee
- Handle validation errors
- Verify error messages and states

DELIVERABLES

- Backend integration test suite
- Selenium UI automation test suite
- Test data setup scripts
- Generated test reports (Allure/HTML)
- Screenshots on failure
- Lab summary and key learnings

LAB OUTCOMES

- **Confident code changes** with reliable integration tests
- **Stable UI automation** with reduced flakiness
- **Faster feedback** through automated reports
- **Better quality** across the full stack

LAB TASKS AND DELIVERABLES

Implement integration tests for the Spring Boot REST API and automate Angular UI tests with Selenium WebDriver.

LAB TASK	WHAT IT COVERS
1. Set Up Test Environment	Configure the PostgreSQL test database, a Spring Boot test profile, and the Angular test setup.
2. Seed Test Data	Write fixtures or scripts that insert consistent, deterministic test data.
3. Write Backend Integration Tests	Cover Controllers, Services, and Repositories using @SpringBootTest and test slices.
4. Validate API Behavior	Test success, error, and edge cases for REST endpoints, including status codes and payloads.
5. Configure Selenium WebDriver	Set up headless WebDriver, the Page Object Model, and a shared test base class.
6. Automate Angular UI Workflows	Script create, view, update, and delete-employee scenarios end to end.
7. Handle Waits and Errors	Use explicit waits for dynamic elements and API-driven UI updates.
8. Capture Screenshots on Failure	Automatically store a screenshot in target/screenshots on every failing test.
9. Generate and Review Test Reports	Publish Surefire, Allure, and consolidated summary reports for both suites.

KEY TAKEAWAY A lab isn't complete until all nine tasks are green in headless mode, screenshots are captured on every failure, and Allure/HTML reports are generated and submitted -- partial coverage doesn't satisfy the rubric.

LAB TASKS AND DELIVERABLES

In this lab, you will implement integration tests for a Spring Boot REST API and automate Angular UI tests using Selenium WebDriver.

LAB TASKS

1. **Set Up Test Environment**
   Configure PostgreSQL test database, Spring Boot test profile, and Angular test setup.
2. **Seed Test Data**
   Create scripts or fixtures to insert consistent test data.
3. **Write Integration Tests (Backend)**
   Create integration tests for Controllers, Services, and Repositories using @SpringBootTest and test slices.
4. **Validate API Behavior**
   Test success, error, and edge cases for REST endpoints including status codes and response payloads.
5. **Configure Selenium WebDriver**
   Set up WebDriver (headless), Page Object Model, and test base framework.
6. **Automate Angular UI Workflows**
   Automate end-to-end scenarios: create, view, update, delete employee using Angular UI.
7. **Handle Waits and Synchronization**
   Use explicit waits for dynamic elements and API-driven UI updates.
8. **Capture Screenshots on Failure**
   Capture and store screenshots automatically when tests fail.
9. **Generate Test Reports**
   Generate and publish reports for integration and UI tests.

DELIVERABLES

Backend Integration Test Suite
- Controller integration tests
- Service integration tests
- Repository integration tests

Test Data Scripts
- SQL scripts or JPA data loaders
- Reusable test data utilities

UI Automation Test Suite
- Page Object classes
- End-to-end scenario tests
- Reusable test base classes

Screenshots on Failure
- Stored in target/screenshots
- Automatically captured

Test Reports
- Spring Boot test reports (Surefire)
- UI test reports (Allure/HTML)
- Consolidated summary

Artifacts Package
- Compiled test reports
- Screenshots
- Logs and evidence

ENVIRONMENT

Backend	Spring Boot 3.x
Database	PostgreSQL (Test DB)
Frontend	Angular 17+
Test Tool (API)	JUnit 5, Testcontainers
Test Tool (UI)	Selenium WebDriver 4.x
Build Tool	Maven
Report Tool	Allure Report

EVALUATION CRITERIA

- All integration tests pass consistently
- UI test scenarios pass in headless mode
- Edge cases and error scenarios are covered
- Screenshots captured on all failures
- Reports generated and artifacts submitted
- Code is modular, clean, and maintainable

SUCCESS CRITERIA

Reliable Tests
Stable, repeatable tests with minimal flakiness

Complete Coverage
Happy path, negative, and edge cases covered

Quality Reporting
Clear, actionable reports with evidence

Production Ready
Tests can be integrated into CI/CD pipeline

MODULE 19 SUMMARY

The application is now proven at every level -- components together, real infrastructure, and a real user's browser.

KEY CONCEPTS COVERED

Test Pyramid Discipline

Many fast unit tests, fewer integration tests, few slow Selenium tests -- each catching what others can't.

Spring Boot Integration Testing

@SpringBootTest, test slices, and Testcontainers-backed PostgreSQL prove the layers genuinely work together.

Deliberate Test Data & Isolation

Known fixture IDs and correctly-scoped @Transactional rollback keep every test repeatable and independent.

Selenium Fundamentals

WebDriver's navigate/locate/interact/quit lifecycle, driven against a real, rendered Angular application.

Stable Selectors & Explicit Waits

data-testid selectors and WebDriverWait-based conditions replace fragile selectors and blind sleeps.

GitHub Actions CI Gate

Headless Selenium runs on every pull request, with screenshots and reports captured automatically on failure.

MODULE FLOW RECAP

1. **Test the Layers**
2. **Choose the DB Strategy**
3. **Automate the UI**
4. **Wait, Don't Sleep**
5. **Gate CI on Green**

KEY TAKEAWAY Good automation is reliable, maintainable, and provides confidence in every release -- integration tests prove the Spring Boot layers fit together, and a small, disciplined Selenium suite proves the Angular UI a real user sees actually works, gated automatically on every GitHub Actions run.

Module Summary

In this module, you learned how to design RESTful APIs using Java by applying REST architectural principles, HTTP standards, and best practices for building scalable, maintainable enterprise APIs.

REST Foundations

Understood REST architectural style, its constraints, and how it enables scalable and interoperable systems.

Resource-Based Design

Learned to identify resources and design clean, intuitive, and consistent URIs aligned with REST principles.

HTTP Mastery

Explored HTTP methods, request/response structure, status codes, and how they power REST communication.

Payloads & Error Handling

Designed JSON requests and responses, handled content negotiation, pagination, filtering, sorting, and error formats.

API Documentation & Best Practices

Used OpenAPI for documentation and applied industry best practices to deliver high-quality, enterprise-ready APIs.

What You Can Do Now

- Design resource-based REST APIs with clean and consistent URIs.
- Choose the right HTTP methods and status codes for every operation.
- Structure JSON requests and responses effectively.
- Implement pagination, filtering, and sorting in your APIs.
- Apply versioning strategies to support API evolution.
- Create OpenAPI documentation for your APIs.
- Follow REST API design best practices and avoid common mistakes.

Key Takeaway: A well-designed REST API is the foundation for reliable, scalable, and secure communication between systems and user interfaces.

KNOWLEDGE CHECK (1 OF 2)

Test your understanding of integration testing, Spring Test annotations, and the test pyramid.

1. A repository test passes against H2 but fails against PostgreSQL in production. What does this most directly indicate?

A. H2 is close enough -- ignore the discrepancy

B. The test should have used a real PostgreSQL instance (e.g. via Testcontainers) from the start

C. PostgreSQL is broken and needs a patch

D. Repository tests should never touch a real database at all

2. Why does @SpringBootTest with RANDOM_PORT plus TestRestTemplate provide stronger confidence than a MockMvc-based test alone?

A. It runs faster than MockMvc

B. It starts a real embedded server and sends genuine HTTP requests, exercising the full stack a client would experience

C. It does not require Spring context to load at all

D. It replaces the need for unit tests entirely

3. Why can @Transactional rollback-based test isolation hide a real production bug?

A. It makes tests run slower than they should

B. The transaction never actually commits, so commit-time behavior (e.g. certain constraints) never gets exercised

C. It deletes the database schema after every test

D. It is incompatible with @SpringBootTest

4. What is the main reason a Selenium suite should stay small, per the test pyramid?

A. Selenium cannot test Angular applications reliably

B. E2E tests are the slowest and most expensive to maintain -- reserve them for critical journeys only

C. GitHub Actions does not support running browser tests

D. Selenium tests cannot run in headless mode

KEY TAKEAWAY Target real PostgreSQL, not just H2; RANDOM_PORT + TestRestTemplate proves a real HTTP round trip; @Transactional rollback can hide commit-time bugs; and Selenium stays small since E2E is the slowest layer.

KNOWLEDGE CHECK (2 OF 2)

Four more questions on stable selectors, explicit waits, flaky tests, and CI artifacts.

5. Why should a Selenium test targeting this course's Angular app prefer a data-testid selector over a generated CSS class?

A. data-testid selectors run faster in the browser

B. Generated classes (e.g. from Angular Material) can change on any dependency upgrade, unrelated to a real bug

C. CSS classes are not supported by the By locator API

D. data-testid is required by the W3C WebDriver protocol

6. Why must a Selenium test wait explicitly after clicking a button that saves a customer, rather than asserting immediately?

A. Selenium's click() method is asynchronous and returns before the click registers

B. The save action triggers an HTTP call to Spring Boot, and the resulting UI update only happens after the response arrives

C. Angular requires a manual page refresh after every button click

D. WebDriver cannot detect DOM changes without an explicit wait call first

7. A Selenium test passes locally but fails intermittently in GitHub Actions with a NoSuchElementException. What is the most likely cause?

A. GitHub Actions does not support Selenium

B. A missing or inadequate explicit wait is racing against Angular's real, variable rendering time

C. The test file was not properly checked into git

D. Headless Chrome cannot render Angular applications

8. Why should a failing GitHub Actions Selenium run always upload a screenshot and test report as artifacts?

A. It is required by the W3C WebDriver protocol

B. It lets a developer diagnose the failure directly from the CI run, without reproducing it locally first

C. It makes the workflow run faster on the next attempt

D. Artifacts are only useful for successful runs, not failures

KEY TAKEAWAY Stable selectors survive incidental CSS/framework changes; explicit waits account for Angular's real async rendering; missing waits are the most common cause of CI-only flakiness; and CI artifacts let a failure be diagnosed without local reproduction.

KNOWLEDGE CHECK

Test your understanding of key REST API design concepts.

Choose the best answer for each question.

1 **What is the primary goal of REST?**

A. To secure data transmission
B. To enable communication between systems using HTTP
C. To replace SOAP web services
D. To enforce a specific tech stack

2 **Which HTTP method is used to create a new resource?**

A. GET
B. PUT
C. POST
D. DELETE

3 **What does a 404 status code indicate?**

A. Internal server error
B. Unauthorized access
C. Resource not found
D. Bad request

4 **Which constraint ensures each request from client contains all necessary information?**

A. Cacheability
B. Statelessness
C. Uniform Interface
D. Layered System

5 **What is the best practice for designing resource URIs?**

A. Use verbs in the URI
B. Use nouns and be resource-oriented
C. Use action-based paths
D. Include file extensions

6 **Which header is used by the client to specify the expected response format?**

A. Content-Type
B. Accept
C. Authorization
D. Cache-Control

Review Key Concepts: REST constraints, HTTP methods, status codes, URI design, headers, pagination, filtering, sorting, and versioning.

Keep Learning!
Strong API design leads to scalable, maintainable, and future-ready systems.

MODULE 19 COMPLETE

"Good automation is reliable, maintainable, and provides confidence in every release."

Integration Testing and UI Test Automation

You can now prove the Spring Boot layers work together and automate real Angular user journeys with a reliable, CI-gated Selenium suite.